# Spring Boot Developer Book

A comprehensive, chapter-wise guide to mastering Spring Boot — from first application to production-grade microservices, security, monitoring, and cloud deployment.

---

## Table of Contents


| # | Chapter | Topics Covered |
|---|---|---|
| 01 | **Introduction to Spring Boot** | What is Spring Boot, why it exists, Spring vs Spring Boot, key features, auto-configuration |
| 02 | **Setup & Installation** | Spring Initializr, IDE setup, Maven vs Gradle, project structure, first REST endpoint |
| 03 | **Core Concepts** | IoC container, Dependency Injection, Beans, Application Context, Component scan |
| 04 | **REST APIs** | @RestController, @RequestMapping, HTTP methods, request/response, DTOs, validation |
| 05 | **Spring Data JPA & Databases** | JPA entities, repositories, CRUD, relationships, JPQL, pagination, transactions |
| 06 | **Spring Security** | Authentication, authorization, JWT, OAuth2, form login, method-level security |
| 07 | **Configuration & Profiles** | application.properties/yaml, profiles, externalized config, @ConfigurationProperties |
| 08 | **Testing Spring Boot** | Unit tests, @SpringBootTest, MockMvc, @DataJpaTest, Testcontainers, test slices |
| 09 | **Actuator & Monitoring** | Health checks, metrics, Prometheus, Grafana, custom endpoints, info endpoint |
| 10 | **Exception Handling & Validation** | @ExceptionHandler, @ControllerAdvice, Bean Validation, custom validators, error responses |

| # | Chapter | Topics Covered |
|---|---|---|
| 11 | **Spring Boot with Docker** | Dockerfile, multi-stage builds, docker-compose, buildpacks, environment variables |
| 12 | **Spring Boot with Kubernetes** | Deploying to K8s, ConfigMaps, Secrets, health probes, Helm charts, scaling |
| 13 | **Spring Boot Microservices** | Microservices architecture, service decomposition, inter-service communication, API Gateway |
| 14 | **Spring Cloud** | Eureka, Config Server, Feign, Resilience4j, Circuit Breaker, distributed tracing |
| 15 | **Caching** | @Cacheable, @CacheEvict, Redis cache, Caffeine, cache configuration, TTL |
| 16 | **Messaging — Kafka & RabbitMQ** | Spring Kafka, @KafkaListener, RabbitMQ, AMQP, producers/consumers, message routing |
| 17 | **Scheduling & Async** | @Scheduled, @Async, thread pools, @EnableAsync, virtual threads (Java 21) |
| 18 | **Spring Boot with GraphQL** | GraphQL schema, @QueryMapping, @MutationMapping, subscriptions, N+1 problem |
| 19 | **Performance & Production** | Connection pooling, lazy loading, query optimization, GraalVM native image, health tuning |
| 20 | **Learning Roadmap** | 6-week hands-on plan from zero to production-ready microservices |

---

## How to Read This Book

**If you're brand new to Spring Boot:** Read 1 → 2 → 3 → 4 → 5 in order. This gives you the core mental model.

**If you're building a REST API:** Read 2 → 3 → 4 → 10 → 7.

**If you're securing an application:** Read 6 → 7 → 10.

**If you're going to production:** Read 8 → 9 → 11 → 12 → 19.

**If you're building microservices:** Read 13 → 14 → 16 → 15.

**If you need to deploy on Kubernetes:** Read 11 → 12.

---

## Key Concepts at a Glance

```
Spring Boot Application
├── Auto-Configuration
│   ├── @SpringBootApplication      (entry point)
│   ├── @EnableAutoConfiguration    (magic starter)
│   └── spring.factories / imports  (auto-config registry)
│
├── IoC Container
│   ├── @Component / @Service / @Repository / @Controller
│   ├── @Bean (manual bean definition)
│   └── @Autowired / Constructor Injection
│
├── Web Layer
│   ├── @RestController             (REST endpoints)
│   ├── @RequestMapping / @GetMapping etc.
│   ├── @RequestBody / @PathVariable / @RequestParam
│   └── ResponseEntity<T>
│
├── Data Layer
│   ├── @Entity / @Table            (JPA mapping)
│   ├── JpaRepository<T, ID>        (CRUD + paging)
│   ├── @Transactional              (transaction boundary)
│   └── Spring Data JDBC / R2DBC    (reactive / simple)
│
├── Security
│   ├── SecurityFilterChain         (request filter chain)
│   ├── UserDetailsService          (custom user loading)
│   ├── JWT / OAuth2                (token-based auth)
│   └── @PreAuthorize               (method security)
│
├── Config & Profiles
│   ├── application.yml             (base config)
│   ├── application-{profile}.yml   (profile-specific)
│   └── @ConfigurationProperties    (typed config binding)
│
└── Production Tooling
    ├── Actuator                    (health, metrics, info)
```

```
      ├── Micrometer + Prometheus     (metrics export)
      └── Testcontainers              (real DB in tests)
```

# Chapter 1: Introduction to Spring Boot

## What is Spring Boot?

**Spring Boot** is an opinionated framework built on top of the **Spring Framework** that makes it easy to create stand-alone, production-grade Spring applications with minimal configuration.

> **Analogy:** Spring Framework is like all the ingredients and tools in a professional kitchen. Spring Boot is the same kitchen but with a pre-set menu, pre-measured ingredients, and a head chef who makes sensible decisions automatically — you can still override anything, but the defaults work great out of the box.

---

## Spring Framework vs Spring Boot

| Feature | Spring Framework | Spring Boot |
|---|---|---|
| Configuration | Extensive XML / Java config | Auto-configuration, convention over configuration |
| Server | Deploy WAR to external Tomcat | Embedded Tomcat/Jetty/Undertow |
| Startup time | More boilerplate | Minutes to first running app |
| Dependency management | Manual version management | BOM (Bill of Materials) manages versions |
| Production readiness | Manual setup | Built-in Actuator endpoints |

---

## Why Spring Boot Exists

Before Spring Boot (pre-2014), building a Spring application required:

- Dozens of XML configuration files
- Manual management of compatible dependency versions
- Separate installation and configuration of an application server
- Lots of boilerplate code for common patterns

Spring Boot solved this with three core ideas:

1. **Auto-configuration** — detects what's on the classpath and configures it automatically
2. **Starter dependencies** — curated groups of dependencies with tested compatible versions
3. **Embedded server** — your app ships as a runnable JAR with Tomcat inside

---

## Key Features

### 1. Auto-Configuration

Spring Boot inspects your classpath and automatically configures beans you would otherwise set up manually.

```
You add: spring-boot-starter-data-jpa
Spring Boot automatically configures:
  - DataSource (connection pool)
  - EntityManagerFactory
  - TransactionManager
  - JpaRepositories
```

### 2. Spring Boot Starters

Starters are dependency descriptors — add one, get all related dependencies with compatible versions.

| Starter | What It Includes |
|---|---|
| `spring-boot-starter-web` | Spring MVC, Tomcat, Jackson, Validation |
| `spring-boot-starter-data-jpa` | Hibernate, JPA, Spring Data, JDBC |
| `spring-boot-starter-security` | Spring Security, filter chain |

| Starter | What It Includes |
|---|---|
| `spring-boot-starter-test` | JUnit 5, Mockito, MockMvc, AssertJ |
| `spring-boot-starter-actuator` | Health, metrics, info endpoints |
| `spring-boot-starter-cache` | Spring Cache abstraction |

### 3. Embedded Server

No WAR files, no external application server. Your app runs as:

```
java -jar my-app.jar
```

### 4. Production-Ready Actuator

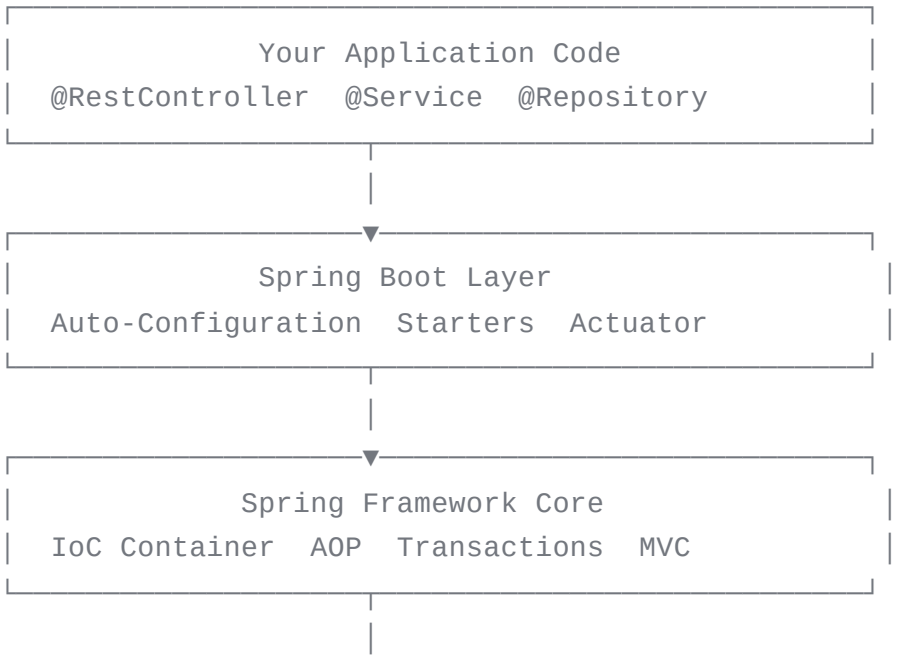
Out-of-the-box endpoints for health checks, metrics, thread dumps, configuration info.

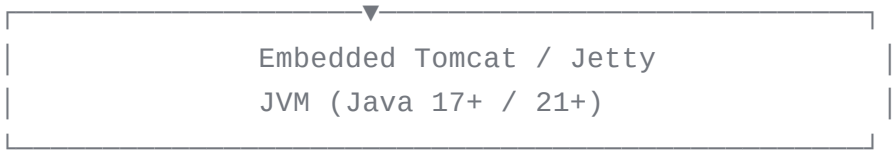
### 5. Externalized Configuration

Configure once, run anywhere — properties change per environment without recompiling.

---

## Spring Boot Architecture Overview






---

## The @SpringBootApplication Annotation

Every Spring Boot app has one entry point:

```java
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

`@SpringBootApplication` is a shortcut for three annotations:

| Annotation | Purpose |
|---|---|
| `@SpringBootConfiguration` | Marks this as a Spring configuration class |
| `@EnableAutoConfiguration` | Triggers Spring Boot's auto-configuration mechanism |
| `@ComponentScan` | Scans the package (and sub-packages) for Spring components |

---

## How Auto-Configuration Works

1. Spring Boot reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
2. For each auto-configuration class, it checks `@ConditionalOn*` conditions:
   - `@ConditionalOnClass` — only configure if this class is on the classpath
   - `@ConditionalOnMissingBean` — only configure if no bean of this type exists yet

- `@ConditionalOnProperty` — only configure if a property is set

3. Matching configurations create beans in the application context

```java
// Example auto-configuration (simplified from Spring Boot source)
@AutoConfiguration
@ConditionalOnClass(DataSource.class)
@ConditionalOnMissingBean(DataSource.class)
public class DataSourceAutoConfiguration {
    @Bean
    public DataSource dataSource(DataSourceProperties properties) {
        return DataSourceBuilder.create()
            .url(properties.getUrl())
            .build();
    }
}
```

---

## Spring Boot Release Timeline

| Version | Java Baseline | Notable Features |
|---|---|---|
| 1.x (2014) | Java 6 | Initial release, embedded server |
| 2.x (2018) | Java 8 | Reactive (WebFlux), Actuator v2, Micrometer |
| 3.x (2022) | Java 17 | GraalVM native images, Jakarta EE 9+, improved observability |
| 3.2+ (2023) | Java 17+ | Virtual thread support (Java 21), RestClient, CDS |

---

## Real-World Use Cases

- **REST APIs & Backend Services** — most common use case, JSON over HTTP
- **Microservices** — small focused services, often deployed in containers
- **Batch Processing** — Spring Batch integration for ETL pipelines

- **Event-Driven Applications** — Kafka/RabbitMQ with Spring Messaging
- **GraphQL APIs** — Spring for GraphQL (since Spring Boot 2.7)
- **Reactive Applications** — Spring WebFlux for non-blocking I/O

---

### Spring Boot vs Alternatives

| Framework | Language | Style | Best For |
|---|---|---|---|
| **Spring Boot** | Java/Kotlin | Opinionated, full-featured | Enterprise apps, large teams |
| **Quarkus** | Java | Cloud-native, fast startup | Serverless, Kubernetes-native |
| **Micronaut** | Java/Kotlin/Groovy | Compile-time DI, fast | Microservices, low overhead |
| **Vert.x** | Java | Reactive, event-driven | High-throughput, async |
| **FastAPI** | Python | Minimal, fast | Python shops, ML APIs |
| **NestJS** | TypeScript | Angular-inspired | Node.js, TypeScript teams |

Spring Boot's strengths: massive ecosystem, battle-tested, extensive documentation, large talent pool.

## Chapter 2: Setup & Installation

### Prerequisites

| Tool | Version | Purpose |
|---|---|---|
| JDK | 17 or 21 (LTS) | Spring Boot 3.x requires Java 17+ |
| Maven or Gradle | Latest | Build tool |
| IDE | IntelliJ IDEA / VS Code / Eclipse | Development environment |

| Tool | Version | Purpose |
|---|---|---|
| Docker | Latest | For containers and Testcontainers |

---

## Option A: Spring Initializr (Recommended)

The fastest way to create a Spring Boot project is at **start.spring.io**.

### Steps:

1. Go to start.spring.io
2. Choose:
   - **Project:** Maven or Gradle
   - **Language:** Java (or Kotlin/Groovy)
   - **Spring Boot version:** Latest stable (3.x)
   - **Group:** `com.example`
   - **Artifact:** `my-app`
   - **Java version:** 17 or 21
3. Add Dependencies: `Spring Web` , `Spring Data JPA` , `H2 Database`
4. Click **Generate** → download ZIP → extract → open in IDE

### Via curl (terminal):

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project \
  -d language=java \
  -d bootVersion=3.3.0 \
  -d baseDir=my-app \
  -d groupId=com.example \
  -d artifactId=my-app \
  -d dependencies=web,data-jpa,h2 \
  -o my-app.zip && unzip my-app.zip
```

---

## Option B: IntelliJ IDEA (Ultimate)

IntelliJ IDEA Ultimate has Spring Initializr built in:

1. **File → New Project → Spring**
2. Fill in project metadata
3. Select dependencies
4. Click **Create**

---

## Project Structure

```
my-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/myapp/
│   │   │       ├── MyAppApplication.java     ← entry point
│   │   │       ├── controller/               ← REST controllers
│   │   │       ├── service/                  ← business logic
│   │   │       ├── repository/               ← data access
│   │   │       └── model/                    ← entities/DTOs
│   │   └── resources/
│   │       ├── application.properties        ← config
│   │       ├── application.yml               ← alternative config (YAML)
│   │       └── static/                       ← static files (HTML, CSS, JS)
│   └── test/
│       └── java/
│           └── com/example/myapp/
│               └── MyAppApplicationTests.java
├── pom.xml                                   ← Maven build file
└── mvnw / gradlew                            ← Maven/Gradle wrapper
```

---

## pom.xml Explained

xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project>
    <modelVersion>4.0.0</modelVersion>
```

```
<!-- Spring Boot parent manages all dependency versions -->
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
</parent>

<groupId>com.example</groupId>
<artifactId>my-app</artifactId>
<version>0.0.1-SNAPSHOT</version>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <!-- Web: REST API support + embedded Tomcat -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- JPA: Hibernate + Spring Data -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- H2: In-memory database (dev only) -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- Testing: JUnit 5 + Mockito + MockMvc -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
```

```
            <!-- Creates executable fat JAR -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

---

## Maven vs Gradle

| Feature | Maven | Gradle |
|---|---|---|
| Config format | XML (pom.xml) | Groovy/Kotlin DSL (build.gradle) |
| Learning curve | Lower | Slightly higher |
| Build speed | Slower | Faster (incremental, parallel) |
| Flexibility | Convention-heavy | Highly customizable |
| Industry adoption | Very common | Growing fast (Android, large projects) |

### Equivalent Gradle (Kotlin DSL):

kotlin

```
// build.gradle.kts
plugins {
    id("org.springframework.boot") version "3.3.0"
    id("io.spring.dependency-management") version "1.1.5"
    kotlin("jvm") version "1.9.24"
    kotlin("plugin.spring") version "1.9.24"
}

dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-data-jpa")
    runtimeOnly("com.h2database:h2")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
}
```

## Running the Application

### With Maven wrapper:

```
./mvnw spring-boot:run
```

### With Gradle wrapper:

```
./gradlew bootRun
```

### Build and run JAR:

```
./mvnw clean package
java -jar target/my-app-0.0.1-SNAPSHOT.jar
```

### With IntelliJ IDEA:

- Click the green play button next to `MyAppApplication.main()`

---

## Your First REST Endpoint

Edit `MyAppApplication.java` or create `HelloController.java` :

java

```
package com.example.myapp.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot!";
```

```
    }
}
```

Run the app and test:

```
curl http://localhost:8080/hello
# Response: Hello, Spring Boot!
```

---

## application.properties vs application.yml

Both are equivalent — use whichever you prefer.

**application.properties:**

```properties
server.port=8080
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

**application.yml (equivalent):**

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
  jpa:
    show-sql: true
    hibernate:
      ddl-auto: update
```

YAML is preferred for complex nested config. Properties is simpler for flat config.

---

## Hot Reload with DevTools

Add to `pom.xml` :

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
</dependency>
```

DevTools provides:

- **Automatic restart** when classpath files change
- **LiveReload** browser refresh for static resources
- **Relaxed property binding** in dev mode

In IntelliJ: Enable "Build project automatically" and "Allow auto-make to start even if developed application is currently running".

---

## Install Java (if needed)

---

### Ubuntu/Debian:

```
sudo apt update
sudo apt install openjdk-21-jdk
java -version
```

### macOS (SDKMAN — recommended):

```
curl -s "https://get.sdkman.io" | bash
sdk install java 21.0.3-tem
sdk use java 21.0.3-tem
java -version
```

### Windows:

Download from adoptium.net and run the installer.

# Chapter 3: Core Concepts — IoC, DI, Beans, and Application Context

## Inversion of Control (IoC)

**IoC** is a design principle where the framework controls the flow of the program, not your code. Instead of you creating objects, the Spring container creates and manages them.

```
Traditional:                        With IoC:
  YourCode → creates → Objects        Spring Container → creates → Objects
  YourCode → wires   → Dependencies   Spring Container → injects → Dependencies
  YourCode → manages → Lifecycle      Spring Container → manages → Lifecycle
```

---

## Dependency Injection (DI)

DI is the mechanism Spring uses to implement IoC. Instead of a class creating its own dependencies, they are **injected** from outside.

### Without DI (tightly coupled):

java

```
public class OrderService {
    private PaymentService paymentService = new PaymentService(); // bad: hard-coded dependency

    public void placeOrder() {
        paymentService.charge();
    }
}
```

### With Constructor Injection (recommended):

java

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) { // Spring injects this
        this.paymentService = paymentService;
```

```
    }

    public void placeOrder() {
        paymentService.charge();
    }
}
```

### Three Types of Injection

| Type | Example | Recommendation |
|---|---|---|
| **Constructor injection** | `public Service(Dep dep)` | **Preferred** — enforces required deps, immutable |
| **Setter injection** | `@Autowired public void setDep(Dep dep)` | For optional dependencies |
| **Field injection** | `@Autowired private Dep dep;` | Avoid — hard to test, hides dependencies |

---

## Spring Beans

A **Bean** is an object that Spring creates, configures, and manages. Every object managed by the Spring IoC container is a bean.

### Declaring Beans

#### Method 1: Stereotype Annotations (auto-detected via component scan):

java

```
@Component          // Generic bean
@Service            // Business logic layer
@Repository         // Data access layer (also translates DB exceptions)
@Controller         // Web layer (returns views)
@RestController     // Web layer (returns data — @Controller + @ResponseBody)
```

java

```
@Service
public class UserService {
    // Spring creates and manages this bean
}
```

**Method 2: @Bean in @Configuration class (explicit):**

```java
@Configuration
public class AppConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Use `@Bean` when:

- You don't own the class (third-party library)
- You need fine-grained control over initialization

---

## Application Context

The **ApplicationContext** is Spring's IoC container. It holds all beans and their relationships.

```java
// Programmatic access (rarely needed in Spring Boot apps)
ApplicationContext ctx = SpringApplication.run(MyApp.class, args);
UserService userService = ctx.getBean(UserService.class);
```

In Spring Boot, the ApplicationContext is created automatically when the app starts. You rarely interact with it directly — just use `@Autowired` or constructor injection.

### Context Lifecycle:

```
Application starts
    ↓
Spring scans @Component, @Service, @Repository, @Controller
    ↓
Spring creates beans in dependency order
    ↓
```

```
Spring injects dependencies
    ↓
@PostConstruct methods run
    ↓
Application is ready → serves requests
    ↓
Application shuts down
    ↓
@PreDestroy methods run
    ↓
Beans destroyed
```

---

## Component Scan

Spring Boot automatically scans the package containing `@SpringBootApplication` and all its sub-packages.

```
com.example.myapp
├── MyAppApplication.java     ← @SpringBootApplication here
├── controller/
│   └── UserController.java   ← auto-detected
├── service/
│   └── UserService.java      ← auto-detected
└── repository/
    └── UserRepository.java   ← auto-detected
```

If you put a class outside this package, you must add it explicitly:

```java
@SpringBootApplication(scanBasePackages = {"com.example.myapp",
"com.other.package"})
```

---

## Bean Scopes

| Scope | Behavior | When to Use |
|---|---|---|
| `singleton` (default) | One instance per container | Stateless services (most beans) |

| Scope | Behavior | When to Use |
|---|---|---|
| `prototype` | New instance every injection | Stateful objects |
| `request` | One per HTTP request | Web-layer beans |
| `session` | One per HTTP session | User session data |

```java
@Component
@Scope("prototype")
public class ReportGenerator {
    // new instance created every time it's injected
}
```

---

## @PostConstruct and @PreDestroy

```java
@Service
public class CacheService {

    @PostConstruct
    public void init() {
        // Runs after bean is created and dependencies are injected
        loadCacheFromDatabase();
    }

    @PreDestroy
    public void cleanup() {
        // Runs before bean is destroyed (app shutdown)
        flushCacheToDisk();
    }
}
```

---

## Qualifiers — Resolving Multiple Beans

When two beans of the same type exist, Spring doesn't know which to inject:

```java
@Service
public class EmailNotifier implements Notifier { ... }

@Service
public class SmsNotifier implements Notifier { ... }
```

### Solution 1: @Qualifier

```java
@Service
public class OrderService {
    public OrderService(@Qualifier("emailNotifier") Notifier notifier) {
        this.notifier = notifier;
    }
}
```

### Solution 2: @Primary

```java
@Service
@Primary   // this one is injected by default
public class EmailNotifier implements Notifier { ... }
```

---

## @Conditional Beans

Create beans only under certain conditions:

```java
@Configuration
public class DataSourceConfig {

    @Bean
    @Profile("dev")
    public DataSource h2DataSource() {
        return new EmbeddedDatabaseBuilder().setType(H2).build();
    }

    @Bean
    @Profile("prod")
    public DataSource postgresDataSource() {
        return DataSourceBuilder.create().url("jdbc:postgresql://...").build();
    }
}
```

Other conditions:

```java
@ConditionalOnProperty(name = "feature.cache.enabled", havingValue = "true")
@ConditionalOnClass(name = "redis.clients.jedis.Jedis")
@ConditionalOnMissingBean(CacheManager.class)
```

---

## Aspect-Oriented Programming (AOP)

AOP lets you add cross-cutting concerns (logging, transactions, security) without modifying business code.

```java
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.example.myapp.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws
Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed();
        long duration = System.currentTimeMillis() - start;
        log.info("{} executed in {}ms", joinPoint.getSignature(), duration);
        return result;
    }
}
```

Spring uses AOP internally for:

- `@Transactional` — wraps methods in database transactions
- `@Cacheable` — intercepts methods to return cached results
- `@PreAuthorize` — checks security before method execution

---

## Summary: Spring Core Mental Model

```
@SpringBootApplication
    ↓ triggers
Component Scan + Auto-Configuration
```

```
    ↓ creates
ApplicationContext (IoC Container)
    ↓ manages
Beans (@Component / @Service / @Repository / @Controller / @Bean)
    ↓ wired via
Dependency Injection (Constructor / Setter / Field)
    ↓ enhanced by
AOP (@Transactional, @Cacheable, @PreAuthorize)
```

# Chapter 4: REST APIs with Spring Boot

## Building REST APIs

Spring Boot's `spring-boot-starter-web` gives you everything to build RESTful APIs:

- **Spring MVC** for routing and request handling
- **Jackson** for JSON serialization/deserialization
- **Embedded Tomcat** as the server

---

## Core Annotations

| Annotation | Purpose |
|---|---|
| `@RestController` | Marks a class as a REST controller (combines @Controller + @ResponseBody) |
| `@RequestMapping` | Maps URL path to a class or method |
| `@GetMapping` | HTTP GET handler |
| `@PostMapping` | HTTP POST handler |
| `@PutMapping` | HTTP PUT handler |
| `@PatchMapping` | HTTP PATCH handler |
| `@DeleteMapping` | HTTP DELETE handler |

| Annotation | Purpose |
|---|---|
| `@PathVariable` | Extracts value from URL path |
| `@RequestParam` | Extracts query parameter |
| `@RequestBody` | Deserializes request body from JSON |
| `@ResponseStatus` | Sets HTTP status code on method/exception |

---

## A Complete CRUD Controller

java

```java
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    // GET /api/users
    @GetMapping
    public List<UserDto> getAllUsers() {
        return userService.findAll();
    }

    // GET /api/users/42
    @GetMapping("/{id}")
    public ResponseEntity<UserDto> getUserById(@PathVariable Long id) {
        return userService.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    // GET /api/users?page=0&size=10&sort=name
    @GetMapping("/search")
    public Page<UserDto> searchUsers(
            @RequestParam(defaultValue = "") String name,
```

```
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {
        return userService.search(name, PageRequest.of(page, size));
    }

    // POST /api/users
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public UserDto createUser(@Valid @RequestBody CreateUserRequest request) {
        return userService.create(request);
    }

    // PUT /api/users/42
    @PutMapping("/{id}")
    public ResponseEntity<UserDto> updateUser(
            @PathVariable Long id,
            @Valid @RequestBody UpdateUserRequest request) {
        return userService.update(id, request)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    // DELETE /api/users/42
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        if (userService.delete(id)) {
            return ResponseEntity.noContent().build();
        }
        return ResponseEntity.notFound().build();
    }
}
```

---

## DTOs (Data Transfer Objects)

Never expose your `@Entity` classes directly in API responses. Use DTOs to:

- Control what fields are exposed
- Decouple API contract from database schema
- Add validation annotations

```java
// Request DTO
public record CreateUserRequest(
    @NotBlank @Size(max = 100) String name,
    @Email @NotBlank String email,
    @NotBlank @Size(min = 8) String password
) {}

// Response DTO
public record UserDto(
    Long id,
    String name,
    String email,
    LocalDateTime createdAt
) {}
```

**Mapping entity to DTO:**

```java
@Service
public class UserService {
    private UserDto toDto(User user) {
        return new UserDto(user.getId(), user.getName(), user.getEmail(),
user.getCreatedAt());
    }
}
```

For larger projects, consider **MapStruct** to auto-generate mappers at compile time.

---

## ResponseEntity — Full Control Over Responses

`ResponseEntity<T>` lets you control status code, headers, and body:

```java
@GetMapping("/{id}")
public ResponseEntity<UserDto> getUser(@PathVariable Long id) {
    return userService.findById(id)
        .map(user -> ResponseEntity
            .ok()
            .header("X-User-Found", "true")
            .body(user))
        .orElse(ResponseEntity.notFound().build());
}
```

```
@PostMapping
public ResponseEntity<UserDto> createUser(@RequestBody CreateUserRequest req) {
    UserDto created = userService.create(req);
    URI location = URI.create("/api/users/" + created.id());
    return ResponseEntity.created(location).body(created);
}
```

---

## HTTP Status Codes Cheat Sheet

| Status | Meaning | When to Use |
|---|---|---|
| `200 OK` | Success | GET, PUT, PATCH success |
| `201 Created` | Resource created | POST success |
| `204 No Content` | Success, no body | DELETE success |
| `400 Bad Request` | Invalid input | Validation failure |
| `401 Unauthorized` | Not authenticated | Missing/invalid token |
| `403 Forbidden` | Not authorized | Authenticated but no permission |
| `404 Not Found` | Resource missing | Entity not found |
| `409 Conflict` | State conflict | Duplicate resource |
| `422 Unprocessable Entity` | Semantic validation error | Business rule violation |
| `500 Internal Server Error` | Server error | Unexpected exception |

---

## Request Validation with Bean Validation

Add `spring-boot-starter-validation` to enable Bean Validation (Jakarta Validation API):

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```java
public record CreateUserRequest(
    @NotBlank(message = "Name is required")
    @Size(max = 100, message = "Name must be under 100 characters")
    String name,

    @Email(message = "Invalid email format")
    @NotBlank
    String email,

    @Min(value = 0, message = "Age must be positive")
    @Max(value = 150)
    Integer age,

    @Pattern(regexp = "^[A-Z]{2}$", message = "Country code must be 2 uppercase letters")
    String countryCode
) {}
```

Use `@Valid` on the controller parameter to trigger validation:

```java
@PostMapping
public UserDto create(@Valid @RequestBody CreateUserRequest request) { ... }
```

Validation failures throw `MethodArgumentNotValidException` — handle it in `@ControllerAdvice` (see Chapter 10).

---

## Path Variables vs Request Params

```
GET /api/users/42           → @PathVariable Long id = 42
GET /api/users?role=ADMIN   → @RequestParam String role = "ADMIN"
GET /api/users/42/posts     → nested resource
```

**Rule of thumb:**

- Use **path variables** for resource identity ( `/users/42` )

- Use **request params** for filtering, sorting, pagination ( `?sort=name&page=2` )

---

## Consuming External REST APIs (RestTemplate / RestClient)

### RestClient (Spring Boot 3.2+ — recommended):

java
```java
@Service
public class GithubService {

    private final RestClient restClient;

    public GithubService(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.github.com")
            .defaultHeader("Accept", "application/vnd.github.v3+json")
            .build();
    }

    public GithubUser getUser(String username) {
        return restClient.get()
            .uri("/users/{username}", username)
            .retrieve()
            .body(GithubUser.class);
    }
}
```

### WebClient (reactive / non-blocking):

java
```java
WebClient webClient = WebClient.builder()
    .baseUrl("https://api.github.com")
    .build();

Mono<GithubUser> user = webClient.get()
    .uri("/users/{username}", username)
    .retrieve()
    .bodyToMono(GithubUser.class);
```

---

## Content Negotiation

Spring Boot auto-configures JSON (Jackson). For XML support:

```xml
<dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

Clients request format via `Accept` header:

```
Accept: application/json   → JSON response
Accept: application/xml    → XML response
```

---

## CORS Configuration

```java
@Configuration
public class CorsConfig {

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("https://myfrontend.com"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }
}
```

Or per-controller:

```java
@RestController
@CrossOrigin(origins = "https://myfrontend.com")
public class UserController { ... }
```

## API Versioning Strategies

| Strategy | Example | Pros/Cons |
|---|---|---|
| **URI versioning** | `/api/v1/users` | Simple, visible, most common |
| **Header versioning** | `Accept: application/vnd.myapi.v1+json` | Clean URLs, harder to test |
| **Query param** | `/api/users?version=1` | Easy but pollutes URL |

```java
// URI versioning example
@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 { ... }

@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 { ... }
```

## Useful Jackson Annotations

```java
public class UserDto {
    @JsonProperty("user_id")       // rename field in JSON
    private Long id;

    @JsonIgnore                    // exclude from serialization
    private String password;

    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private LocalDateTime createdAt;

    @JsonInclude(JsonInclude.Include.NON_NULL)  // omit null fields
    private String middleName;
}
```

Global Jackson config in `application.yml` :

```yaml
spring:
  jackson:
    default-property-inclusion: non_null
    serialization:
      write-dates-as-timestamps: false
    deserialization:
      fail-on-unknown-properties: false
```

# Chapter 5: Spring Data JPA & Databases

## What is JPA?

**JPA (Jakarta Persistence API)** is a specification for mapping Java objects to relational database tables (ORM). **Hibernate** is the most popular JPA implementation. **Spring Data JPA** adds a repository abstraction on top of JPA that eliminates boilerplate CRUD code.

```
Your Code → Spring Data JPA → JPA (Hibernate) → JDBC → Database
```

---

## Dependencies

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

---

## Database Configuration

```yaml
# application.yml
spring:
  datasource:
```

```
    url: jdbc:postgresql://localhost:5432/mydb
    username: myuser
    password: mypassword
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 2

  jpa:
    hibernate:
      ddl-auto: validate        # use 'create-drop' in dev, 'validate' in prod
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

**ddl-auto values:**

| Value | Behavior |
|---|---|
| `none` | No action |
| `validate` | Validates schema matches entities (prod) |
| `update` | Alters tables to match entities (dev) |
| `create` | Drops and recreates on every start |
| `create-drop` | Creates on start, drops on stop (tests) |

---

## Defining Entities

java

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, length = 100)
    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Enumerated(EnumType.STRING)
    private UserStatus status;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // getters, setters (or use Lombok)
}
```

### Use Lombok to reduce boilerplate:

java

```
@Entity
@Table(name = "users")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
}
```

---

## Spring Data Repositories

Spring Data JPA generates implementations automatically at runtime.

```java
// Basic CRUD
public interface UserRepository extends JpaRepository<User, Long> {
    // JpaRepository gives you: save, findById, findAll, delete, count, etc.
}
```

## Derived Query Methods

Spring Data parses method names and generates SQL:

```java
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    List<User> findByNameContainingIgnoreCase(String name);

    List<User> findByStatusAndCreatedAtAfter(UserStatus status, LocalDateTime date);

    boolean existsByEmail(String email);

    long countByStatus(UserStatus status);

    void deleteByEmail(String email);

    List<User> findTop10ByOrderByCreatedAtDesc();
}
```

### Keyword mapping:

| Keyword | SQL | Example |
|---|---|---|
| findBy | WHERE | findByName |
| And | AND | findByNameAndEmail |
| Or | OR | findByNameOrEmail |
| Containing | LIKE %x% | findByNameContaining |
| StartingWith | LIKE x% | findByNameStartingWith |
| Between | BETWEEN | findByAgeBetween |

| Keyword | SQL | Example |
|---|---|---|
| LessThan | < | findByAgeLessThan |
| GreaterThan | > | findByAgeGreaterThan |
| IsNull | IS NULL | findByEmailIsNull |
| OrderBy | ORDER BY | findByStatusOrderByNameAsc |
| Top / First | LIMIT | findTop5ByStatus |

---

## Custom JPQL Queries

```java
public interface UserRepository extends JpaRepository<User, Long> {

    // JPQL — uses entity names, not table names
    @Query("SELECT u FROM User u WHERE u.email = :email AND u.status =
:status")
    Optional<User> findActiveByEmail(@Param("email") String email,
                                     @Param("status") UserStatus status);

    // Native SQL — when JPQL isn't expressive enough
    @Query(value = "SELECT * FROM users WHERE created_at > NOW() - INTERVAL '7
days'",
           nativeQuery = true)
    List<User> findRecentUsers();

    // Modifying queries
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.status = :status WHERE u.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") UserStatus status);
}
```

---

## Pagination and Sorting

```java
// Repository
public interface UserRepository extends JpaRepository<User, Long> {
    Page<User> findByStatus(UserStatus status, Pageable pageable);
}

// Service
@Service
public class UserService {
    public Page<UserDto> getUsers(int page, int size, String sortField) {
        Pageable pageable = PageRequest.of(page, size,
Sort.by(sortField).ascending());
        return userRepository.findByStatus(UserStatus.ACTIVE, pageable)
            .map(this::toDto);
    }
}

// Controller
@GetMapping
public Page<UserDto> getUsers(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "10") int size,
        @RequestParam(defaultValue = "name") String sort) {
    return userService.getUsers(page, size, sort);
}
```

Page response includes: `content` , `totalElements` , `totalPages` , `number` , `size` .

---

## Entity Relationships

### One-to-Many / Many-to-One

```java
@Entity
public class Post {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id")
    private User author;
```

```
    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval =
true)
    private List<Comment> comments = new ArrayList<>();
}

@Entity
public class Comment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String content;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id")
    private Post post;
}
```

### Many-to-Many

java

```
@Entity
public class User {
    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();
}
```

---

## Fetch Types and N+1 Problem

`LAZY` = load on demand (default for collections)
`EAGER` = load immediately (default for @ManyToOne)

**N+1 Problem:** Loading 100 posts then querying for each author separately = 101 queries.

**Fix with JOIN FETCH:**

java

```
@Query("SELECT p FROM Post p JOIN FETCH p.author WHERE p.status = 'PUBLISHED'")
List<Post> findPublishedPostsWithAuthors();
```

**Fix with @EntityGraph:**

```java
@EntityGraph(attributePaths = {"author", "comments"})
List<Post> findByStatus(String status);
```

---

## @Transactional

```java
@Service
public class OrderService {

    @Transactional
    public Order placeOrder(Long userId, List<Long> itemIds) {
        User user = userRepository.findById(userId).orElseThrow();
        List<Item> items = itemRepository.findAllById(itemIds);

        Order order = new Order(user, items);
        orderRepository.save(order);

        inventoryService.deduct(items);   // if this throws, order is rolled back

        return order;
    }

    @Transactional(readOnly = true)   // optimization: no dirty check, read replica possible
    public List<Order> getOrders(Long userId) {
        return orderRepository.findByUserId(userId);
    }
}
```

| Property | Options |
|---|---|
| `propagation` | `REQUIRED` (default), `REQUIRES_NEW`, `SUPPORTS`, `NEVER` |
| `isolation` | `DEFAULT`, `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE` |
| `rollbackFor` | Exception classes that trigger rollback |
| `readOnly` | `true` for read-only transactions |

## Auditing

Automatically track who created/updated entities:

```java
@Configuration
@EnableJpaAuditing
public class JpaConfig {
    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> Optional.ofNullable(SecurityContextHolder.getContext())
            .map(ctx -> ctx.getAuthentication().getName());
    }
}

@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {
    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @CreatedBy
    @Column(updatable = false)
    private String createdBy;

    @LastModifiedBy
    private String updatedBy;
}
```

## Database Migration with Flyway

Flyway manages database schema changes with versioned SQL scripts. Much better than `ddl-auto=update` in production.

```xml
<dependency>
    <groupId>org.flywaydb</groupId>
```

```
    <artifactId>flyway-core</artifactId>
</dependency>
```

```
src/main/resources/db/migration/
├── V1__create_users_table.sql
├── V2__add_email_index.sql
└── V3__add_roles_table.sql
```

```sql
-- V1__create_users_table.sql
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

Flyway runs missing migrations on startup automatically.

---

### Spring Data with Multiple Databases

```yaml
spring:
  datasource:
    primary:
      url: jdbc:postgresql://primary-host/mydb
    secondary:
      url: jdbc:postgresql://readonly-host/mydb
```

```java
@Configuration
@EnableJpaRepositories(
    basePackages = "com.example.repository.primary",
    entityManagerFactoryRef = "primaryEntityManagerFactory"
)
public class PrimaryDataSourceConfig { ... }
```

## Chapter 6: Spring Security

### What is Spring Security?

**Spring Security** is a powerful, highly customizable authentication and authorization framework for Spring applications. It handles:

- **Authentication** — who are you? (login, JWT, OAuth2)
- **Authorization** — what can you do? (roles, permissions)
- **Protection** — CSRF, session fixation, clickjacking, etc.

---

## Dependency

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Adding this dependency alone immediately:

- Blocks all requests by default (requires authentication)
- Generates a random password printed to the console at startup
- Enables HTTP Basic authentication

---

## Security Filter Chain

Spring Security is a chain of filters that process every HTTP request. You configure it with a `SecurityFilterChain` bean:

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)          // disable for REST
APIs
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // no
```

```
    sessions for JWT
                .authorizeHttpRequests(auth -> auth
                    .requestMatchers("/api/auth/**").permitAll()              //
    public endpoints
                    .requestMatchers("/api/admin/**").hasRole("ADMIN")        //
    admin only
                    .requestMatchers(HttpMethod.GET,
    "/api/products/**").permitAll()
                    .anyRequest().authenticated()                             //
    all else need auth
                )
                .addFilterBefore(jwtAuthFilter,
    UsernamePasswordAuthenticationFilter.class);

            return http.build();
        }
    }
```

---

## JWT Authentication (Most Common for REST APIs)

---

### Flow:

```
    Client → POST /api/auth/login {username, password}
           ← 200 OK { token: "eyJhbGci..." }

    Client → GET /api/users (Authorization: Bearer eyJhbGci...)
           → JwtAuthFilter validates token
           → SecurityContext populated with user
           ← 200 OK [users...]
```

### Dependencies:

xml

```
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.5</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
```

```
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.5</version>
        <scope>runtime</scope>
    </dependency>
```

### JWT Service:

java

```java
@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secretKey;

    @Value("${app.jwt.expiration-ms:3600000}")
    private long expirationMs;

    public String generateToken(UserDetails userDetails) {
        return Jwts.builder()
            .subject(userDetails.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }

    public String extractUsername(String token) {
        return getClaims(token).getSubject();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        return extractUsername(token).equals(userDetails.getUsername())
            && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return getClaims(token).getExpiration().before(new Date());
    }

    private Claims getClaims(String token) {
```

```
        return
Jwts.parser().verifyWith(getSigningKey()).build().parseSignedClaims(token).getP
ayload();
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretKey));
    }
}
```

## JWT Filter:

java

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws ServletException,
IOException {
        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }

        String token = authHeader.substring(7);
        String username = jwtService.extractUsername(token);

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);
            if (jwtService.isTokenValid(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
```

```
SecurityContextHolder.getContext().setAuthentication(authToken);
            }
        }

        chain.doFilter(request, response);
    }
}
```

### Auth Controller:

```java
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody AuthRequest request)
{
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.username(),
request.password())
        );
        UserDetails userDetails =
userDetailsService.loadUserByUsername(request.username());
        String token = jwtService.generateToken(userDetails);
        return ResponseEntity.ok(new AuthResponse(token));
    }
}
```

---

## Custom UserDetailsService

```java
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
```

```
    public UserDetails loadUserByUsername(String email) throws
UsernameNotFoundException {
        return userRepository.findByEmail(email)
            .map(user ->
org.springframework.security.core.userdetails.User.builder()
                .username(user.getEmail())
                .password(user.getPassword())
                .roles(user.getRole().name())
                .build())
            .orElseThrow(() -> new UsernameNotFoundException("User not found: "
+ email));
    }
}
```

---

## Password Encoding

**Never store plain-text passwords.**

java

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();  // uses bcrypt with cost factor 10
}

// In registration:
user.setPassword(passwordEncoder.encode(rawPassword));

// Spring Security compares automatically during authentication
```

---

## Role-Based Authorization

java

```
// In SecurityFilterChain:
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/manager/**").hasAnyRole("ADMIN", "MANAGER")
    .anyRequest().authenticated()
)
```

### Method-Level Security

```java
@Configuration
@EnableMethodSecurity   // must enable this
public class SecurityConfig { ... }

@Service
public class OrderService {

    @PreAuthorize("hasRole('ADMIN') or #userId == authentication.name")
    public List<Order> getOrders(String userId) { ... }

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteOrder(Long id) { ... }

    @PostAuthorize("returnObject.userId == authentication.name")
    public Order getOrder(Long id) { ... }
}
```

---

## OAuth2 Login (Social Login)

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: ${GOOGLE_CLIENT_ID}
            client-secret: ${GOOGLE_CLIENT_SECRET}
            scope: email, profile
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
```

```java
http.oauth2Login(oauth2 -> oauth2
    .loginPage("/login")
    .defaultSuccessUrl("/dashboard")
);
```

---

## Securing Actuator Endpoints

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics
```

```java
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/actuator/health").permitAll()
    .requestMatchers("/actuator/**").hasRole("ADMIN")
    .anyRequest().authenticated()
);
```

---

## CORS + Security

Spring Security overrides any CORS config you set elsewhere. Configure CORS inside the SecurityFilterChain:

```java
http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
```

---

## Common Security Configurations

```java
// Permit static resources
.requestMatchers("/static/**", "/css/**", "/js/**", "/images/**").permitAll()

// H2 console (dev only)
```

```
    .requestMatchers("/h2-console/**").permitAll()
    http.headers(headers -> headers.frameOptions(FrameOptionsConfig::disable));

    // Remember-me
    http.rememberMe(remember -> remember.key("my-secret-
    key").tokenValiditySeconds(86400));
```

---

### Security Best Practices

- Always use HTTPS in production
- Store JWT secret in environment variables, not in code
- Use short-lived access tokens (15 min) + refresh tokens (7 days)
- Never log raw passwords or tokens
- Use `@Transactional(readOnly = true)` for `loadUserByUsername`
- Rate-limit login endpoints to prevent brute force
- Use `BCryptPasswordEncoder` — never MD5/SHA1

## Chapter 7: Configuration & Profiles

### Configuration Sources (Priority Order)

Spring Boot loads configuration from multiple sources. Higher items override lower items:

```
1. Command-line arguments              (--server.port=9090)
2. OS environment variables            (SERVER_PORT=9090)
3. application-{profile}.properties   (profile-specific)
4. application.properties / yml       (base config)
5. @PropertySource annotations
6. Default values in @Value
```

---

### application.properties vs application.yml

Both are equivalent. Pick one style and be consistent.

**application.properties:**

```properties
server.port=8080
server.servlet.context-path=/api

spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=myuser
spring.datasource.password=${DB_PASSWORD}

spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=validate

logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG
```

**application.yml (equivalent):**

```yaml
server:
  port: 8080
  servlet:
    context-path: /api

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: myuser
    password: ${DB_PASSWORD}
  jpa:
    show-sql: false
    hibernate:
      ddl-auto: validate

logging:
  level:
    com.example: DEBUG
    org.hibernate.SQL: DEBUG
```

---

## Profiles

Profiles let you have different configurations for different environments.

```
application.yml           ← base config (loaded always)
application-dev.yml       ← dev overrides
application-test.yml      ← test overrides
application-prod.yml      ← production overrides
```

### application-dev.yml:

yaml

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:devdb
  jpa:
    show-sql: true
    hibernate:
      ddl-auto: create-drop

logging:
  level:
    com.example: DEBUG
```

### application-prod.yml:

yaml

```yaml
spring:
  datasource:
    url: ${DATABASE_URL}
    username: ${DATABASE_USER}
    password: ${DATABASE_PASS}
    hikari:
      maximum-pool-size: 20
  jpa:
    show-sql: false
    hibernate:
      ddl-auto: validate

logging:
  level:
    root: WARN
    com.example: INFO
```

### Activating a Profile

```
# Command line
java -jar app.jar --spring.profiles.active=prod

# Environment variable
export SPRING_PROFILES_ACTIVE=prod
java -jar app.jar

# In application.yml
spring:
  profiles:
    active: dev   # default profile
```

### In tests:

```java
@SpringBootTest
@ActiveProfiles("test")
class UserServiceTest { ... }
```

---

## @ConfigurationProperties — Typed Configuration

Instead of scattered `@Value` annotations, bind entire config sections to a typed class.

```yaml
# application.yml
app:
  mail:
    host: smtp.gmail.com
    port: 587
    username: noreply@example.com
    password: ${MAIL_PASSWORD}
    from-name: My App
  jwt:
    secret: ${JWT_SECRET}
    expiration-ms: 3600000
  feature:
    signup-enabled: true
    max-users-per-org: 100
```

```java
@ConfigurationProperties(prefix = "app.mail")
@Validated   // enables Bean Validation on config fields
```

```
public record MailProperties(
    @NotBlank String host,
    @Min(1) @Max(65535) int port,
    @Email String username,
    @NotBlank String password,
    String fromName
) {}

@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(
    @NotBlank String secret,
    @Positive long expirationMs
) {}
```

Enable with `@EnableConfigurationProperties` or just add `@Component` :

java

```
@Configuration
@EnableConfigurationProperties({MailProperties.class, JwtProperties.class})
public class AppConfig {}
```

Use them anywhere via injection:

java

```
@Service
public class EmailService {
    private final MailProperties mail;

    public EmailService(MailProperties mail) {
        this.mail = mail;
    }
}
```

---

## @Value Annotation

For simple single-value injection:

java

```
@Service
public class MyService {

    @Value("${app.name:My Default App}")    // with default value
    private String appName;
```

```
    @Value("${server.port}")
    private int port;

    @Value("${feature.flags}")              // comma-separated → List
    private List<String> featureFlags;

    @Value("#{systemProperties['user.home']}") // Spring Expression Language
    private String userHome;
}
```

Prefer `@ConfigurationProperties` over `@Value` for groups of related config.

---

## Externalized Configuration (12-Factor App)

In production, configuration should come from environment variables — never hardcoded or committed to git.

yaml

```
# application-prod.yml — references env vars
spring:
  datasource:
    url: ${DATABASE_URL}
    username: ${DATABASE_USER}
    password: ${DATABASE_PASS}

app:
  jwt:
    secret: ${JWT_SECRET}
  stripe:
    api-key: ${STRIPE_API_KEY}
```

### In Docker / Kubernetes:

```
# Docker
docker run -e DATABASE_URL=jdbc:postgresql://db/mydb \
           -e DATABASE_USER=admin \
           -e DATABASE_PASS=secret \
           my-app.jar

# Kubernetes Secret + env ref
env:
  - name: DATABASE_PASS
```

```
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

---

## Relaxed Binding

Spring Boot accepts config keys in multiple formats — they all map to the same property:

```
# All equivalent:
spring.datasource.driver-class-name=...
spring.datasource.driverClassName=...
SPRING_DATASOURCE_DRIVER_CLASS_NAME=...       ← env var
--spring.datasource.driver-class-name=...     ← CLI arg
```

---

## Configuration Metadata (IDE Support)

Create `META-INF/spring-configuration-metadata.json` or use the annotation processor:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional>
</dependency>
```

This generates metadata for `@ConfigurationProperties` classes, enabling IDE autocomplete in `application.yml`.

---

## Logging Configuration

```yaml
logging:
  level:
    root: INFO
    com.example.myapp: DEBUG
```

```
    org.springframework.security: DEBUG
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE    # logs parameter values
  file:
    name: logs/application.log
  pattern:
    console: "%d{HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"
```

### Custom Logback config ( `logback-spring.xml` ):

xml

```
<configuration>
    <springProfile name="prod">
        <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
            <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
        </appender>
        <root level="INFO">
            <appender-ref ref="JSON"/>
        </root>
    </springProfile>
</configuration>
```

---

## Profile-Specific Beans

java

```
@Configuration
public class StorageConfig {

    @Bean
    @Profile("dev")
    public StorageService localStorageService() {
        return new LocalFileStorageService("/tmp/uploads");
    }

    @Bean
    @Profile("prod")
    public StorageService s3StorageService(S3Client s3Client) {
        return new S3StorageService(s3Client, "my-s3-bucket");
    }
}
```

### Config Server (Centralized Configuration)

For microservices, use **Spring Cloud Config Server** to serve config from a Git repository:

```yaml
# bootstrap.yml (on each microservice)
spring:
  config:
    import: configserver:http://config-server:8888
  application:
    name: user-service
  profiles:
    active: prod
```

The config server fetches `user-service-prod.yml` from Git and serves it to the app.

## Chapter 8: Testing Spring Boot Applications

### Testing Philosophy

Spring Boot supports multiple testing approaches at different speeds and scope:

```
Unit Tests          → fast, no Spring context, mock dependencies
Slice Tests         → load partial context (@WebMvcTest, @DataJpaTest)
Integration Tests   → full Spring context (@SpringBootTest)
End-to-End Tests    → full stack with real DB (Testcontainers)
```

---

### Dependencies

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

This includes: JUnit 5, Mockito, MockMvc, AssertJ, Hamcrest, Testcontainers support.

---

## Unit Tests (No Spring Context)

Test pure business logic without starting Spring:

```java
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private EmailService emailService;

    @InjectMocks
    private UserService userService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    void createUser_ShouldReturnDto_WhenEmailIsUnique() {
        // Given
        CreateUserRequest request = new CreateUserRequest("Alice",
"alice@example.com");
        User savedUser = new User(1L, "Alice", "alice@example.com");

when(userRepository.existsByEmail("alice@example.com")).thenReturn(false);
        when(userRepository.save(any(User.class))).thenReturn(savedUser);

        // When
        UserDto result = userService.create(request);

        // Then
        assertThat(result.name()).isEqualTo("Alice");
        assertThat(result.email()).isEqualTo("alice@example.com");
        verify(emailService).sendWelcomeEmail("alice@example.com");
    }

    @Test
    void createUser_ShouldThrow_WhenEmailAlreadyExists() {

when(userRepository.existsByEmail("alice@example.com")).thenReturn(true);
        CreateUserRequest request = new CreateUserRequest("Alice",
"alice@example.com");
```

```
        assertThatThrownBy(() -> userService.create(request))
            .isInstanceOf(DuplicateEmailException.class)
            .hasMessageContaining("alice@example.com");
    }
}
```

### Use @ExtendWith(MockitoExtension.class) to remove boilerplate:

```java
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    // no setUp needed
}
```

---

## @WebMvcTest — Controller Layer Tests

Loads only the web layer (controllers, filters, exception handlers). No real database.

```java
@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean    // creates a Mockito mock and adds it to Spring context
    private UserService userService;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    void getUser_ShouldReturn200_WhenUserExists() throws Exception {
        UserDto user = new UserDto(1L, "Alice", "alice@example.com",
```

```
LocalDateTime.now());
        when(userService.findById(1L)).thenReturn(Optional.of(user));

        mockMvc.perform(get("/api/users/1")
                .contentType(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Alice"))
            .andExpect(jsonPath("$.email").value("alice@example.com"));
    }

    @Test
    void createUser_ShouldReturn400_WhenEmailIsInvalid() throws Exception {
        CreateUserRequest request = new CreateUserRequest("Alice", "not-an-
email");

        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest());
    }

    @Test
    void createUser_ShouldReturn201_WithLocationHeader() throws Exception {
        CreateUserRequest request = new CreateUserRequest("Alice",
"alice@example.com");
        UserDto created = new UserDto(1L, "Alice", "alice@example.com",
LocalDateTime.now());
        when(userService.create(any())).thenReturn(created);

        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location",
containsString("/api/users/1")));
    }
}
```

---

## @DataJpaTest — Repository Layer Tests

Loads only JPA components with an in-memory H2 database:

java

```java
@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private TestEntityManager entityManager;

    @Test
    void findByEmail_ShouldReturnUser_WhenExists() {
        // Setup test data
        User user = new User();
        user.setName("Alice");
        user.setEmail("alice@example.com");
        entityManager.persistAndFlush(user);

        // Test
        Optional<User> found = userRepository.findByEmail("alice@example.com");
        assertThat(found).isPresent();
        assertThat(found.get().getName()).isEqualTo("Alice");
    }

    @Test
    void findByStatus_WithPagination_ShouldReturnPage() {
        // create 15 users
        IntStream.range(1, 16).forEach(i -> {
            User u = new User("User" + i, "user" + i + "@example.com",
UserStatus.ACTIVE);
            entityManager.persist(u);
        });
        entityManager.flush();

        Page<User> page = userRepository.findByStatus(
            UserStatus.ACTIVE,
            PageRequest.of(0, 10)
        );

        assertThat(page.getTotalElements()).isEqualTo(15);
        assertThat(page.getContent()).hasSize(10);
    }
}
```

## @SpringBootTest — Full Integration Tests

Loads the entire Spring context:

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class UserIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private UserRepository userRepository;

    @BeforeEach
    void setUp() {
        userRepository.deleteAll();
    }

    @Test
    void fullUserLifecycle() {
        // Create
        CreateUserRequest createReq = new CreateUserRequest("Alice",
"alice@example.com");
        ResponseEntity<UserDto> createResponse = restTemplate.postForEntity(
            "/api/users", createReq, UserDto.class);

assertThat(createResponse.getStatusCode()).isEqualTo(HttpStatus.CREATED);

        Long userId = createResponse.getBody().id();

        // Read
        ResponseEntity<UserDto> getResponse = restTemplate.getForEntity(
            "/api/users/" + userId, UserDto.class);
        assertThat(getResponse.getStatusCode()).isEqualTo(HttpStatus.OK);
        assertThat(getResponse.getBody().name()).isEqualTo("Alice");

        // Delete
        restTemplate.delete("/api/users/" + userId);
        assertThat(userRepository.existsById(userId)).isFalse();
    }
}
```

## Testcontainers — Real Database in Tests

Use real PostgreSQL/MySQL/Redis in tests via Docker containers:

xml
```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-testcontainers</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <scope>test</scope>
</dependency>
```

java
```java
@SpringBootTest
@Testcontainers
class UserRepositoryIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16-alpine")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldPersistUser() {
        User user = new User("Alice", "alice@example.com");
        userRepository.save(user);

assertThat(userRepository.findByEmail("alice@example.com")).isPresent();
```

```
    }
}
```

Spring Boot 3.1+ supports `@ServiceConnection` for simpler setup:

```java
@Container
@ServiceConnection
static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16-alpine");
// No @DynamicPropertySource needed!
```

---

## Testing with Security

```java
@WebMvcTest(UserController.class)
class SecuredControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    @WithMockUser(username = "alice", roles = "USER")
    void getProfile_ShouldReturn200_ForAuthenticatedUser() throws Exception {
        mockMvc.perform(get("/api/profile"))
            .andExpect(status().isOk());
    }

    @Test
    void getProfile_ShouldReturn401_WhenUnauthenticated() throws Exception {
        mockMvc.perform(get("/api/profile"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void deleteUser_ShouldReturn204_ForAdmin() throws Exception {
        mockMvc.perform(delete("/api/users/1"))
            .andExpect(status().isNoContent());
    }
}
```

## application-test.yml

```yaml
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true

logging:
  level:
    root: WARN
    com.example: INFO
```

## AssertJ Cheat Sheet

```java
// Basic assertions
assertThat(value).isEqualTo(expected);
assertThat(value).isNotNull();
assertThat(value).isNull();
assertThat(list).hasSize(3);
assertThat(list).isEmpty();
assertThat(list).contains("element");
assertThat(list).containsExactly("a", "b", "c");
assertThat(str).startsWith("Hello");
assertThat(str).contains("world");

// Exceptions
assertThatThrownBy(() -> service.method())
    .isInstanceOf(MyException.class)
    .hasMessage("Expected message");

// Optional
assertThat(optional).isPresent();
assertThat(optional).isEmpty();
assertThat(optional).hasValue("expected");
```

### Test Slice Summary

| Annotation | What it loads | Use for |
|---|---|---|
| `@WebMvcTest` | Controllers, filters, security | HTTP layer, request/response |
| `@DataJpaTest` | JPA repositories, H2 | Database queries |
| `@DataMongoTest` | MongoDB repositories | MongoDB queries |
| `@JsonTest` | Jackson serialization | JSON mapping |
| `@RestClientTest` | RestTemplate/RestClient | External HTTP calls |
| `@SpringBootTest` | Full context | Integration / E2E |

## Chapter 9: Actuator & Monitoring

### What is Spring Boot Actuator?

**Spring Boot Actuator** adds production-ready features to your application — health checks, metrics, configuration info, thread dumps, and more — accessible via HTTP endpoints or JMX.

---

### Dependency

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

---

### Built-In Endpoints

| Endpoint | URL | Description |
|---|---|---|
| `health` | `/actuator/health` | Application and component health |
| `info` | `/actuator/info` | App metadata (version, git, etc.) |
| `metrics` | `/actuator/metrics` | JVM, HTTP, custom metrics |
| `env` | `/actuator/env` | All configuration properties |
| `beans` | `/actuator/beans` | All Spring beans |
| `mappings` | `/actuator/mappings` | All @RequestMapping routes |
| `loggers` | `/actuator/loggers` | View/change log levels at runtime |
| `threaddump` | `/actuator/threaddump` | JVM thread dump |
| `heapdump` | `/actuator/heapdump` | JVM heap dump (binary) |
| `shutdown` | `/actuator/shutdown` | Graceful shutdown (disabled by default) |
| `httptrace` | `/actuator/httptrace` | Recent HTTP requests |

---

## Exposing Endpoints

By default, only `health` is exposed over HTTP. Configure exposure:

yaml

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, loggers, env
        # or include: "*" to expose all (only in dev)
      base-path: /actuator     # default
  endpoint:
    health:
      show-details: always     # show component details
    shutdown:
      enabled: true            # enable graceful shutdown
  server:
```

```
    port: 8081                  # separate port for actuator (production best
practice)
```

---

## Health Endpoint

```json
// GET /actuator/health
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": { "database": "PostgreSQL", "validationQuery": "isValid()" }
    },
    "redis": {
      "status": "UP",
      "details": { "version": "7.0.11" }
    },
    "diskSpace": {
      "status": "UP",
      "details": { "total": 499963174912, "free": 291275534336, "threshold":
10485760 }
    }
  }
}
```

### Custom Health Indicator:

```java
@Component
public class ExternalApiHealthIndicator implements HealthIndicator {

    private final ExternalApiClient client;

    @Override
    public Health health() {
        try {
            client.ping();
            return Health.up().withDetail("responseTime", "12ms").build();
        } catch (Exception e) {
            return Health.down()
                .withDetail("error", e.getMessage())
```

```
                .build();
        }
    }
}
```

---

## Info Endpoint

```yaml
management:
  info:
    env:
      enabled: true
    git:
      enabled: true
      mode: full
    build:
      enabled: true

info:
  app:
    name: My Application
    version: '@project.version@'    # pulled from pom.xml
    description: '@project.description@'
  contact:
    team: backend
    email: backend@example.com
```

For git info, add the plugin:

```xml
<plugin>
    <groupId>io.github.git-commit-id</groupId>
    <artifactId>git-commit-id-maven-plugin</artifactId>
</plugin>
```

---

## Metrics

Spring Boot uses **Micrometer** — a metrics facade that supports many backends.

```json
// GET /actuator/metrics
{
  "names": [
    "http.server.requests",
    "jvm.memory.used",
    "jvm.gc.pause",
    "process.cpu.usage",
    "spring.data.repository.invocations",
    "hikaricp.connections.active"
  ]
}

// GET /actuator/metrics/http.server.requests
{
  "name": "http.server.requests",
  "measurements": [
    { "statistic": "COUNT", "value": 1042 },
    { "statistic": "TOTAL_TIME", "value": 5.231 },
    { "statistic": "MAX", "value": 0.152 }
  ]
}
```

### Custom Metrics:

```java
@Service
public class OrderService {

    private final Counter orderCreatedCounter;
    private final Timer orderProcessingTimer;

    public OrderService(MeterRegistry registry) {
        this.orderCreatedCounter = registry.counter("orders.created", "status",
"success");
        this.orderProcessingTimer =
registry.timer("orders.processing.duration");
    }

    public Order placeOrder(OrderRequest request) {
        return orderProcessingTimer.record(() -> {
            Order order = processOrder(request);
            orderCreatedCounter.increment();
            return order;
        });
```

```
    }
}
```

Annotations approach:

```java
@Timed(value = "user.creation.time", description = "Time to create a user")
public UserDto createUser(CreateUserRequest request) { ... }
```

---

## Prometheus + Grafana Integration

### Add Micrometer Prometheus registry:

```xml
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, prometheus
  metrics:
    export:
      prometheus:
        enabled: true
    tags:
      application: ${spring.application.name}
      environment: ${spring.profiles.active}
```

Prometheus scrapes `GET /actuator/prometheus` in Prometheus format.

### Prometheus config ( `prometheus.yml` ):

```yaml
scrape_configs:
  - job_name: 'spring-boot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
```

```
        - targets: ['app:8080']
      scrape_interval: 15s
```

### Docker Compose for full monitoring stack:

```yaml
version: '3.8'
services:
  app:
    image: my-spring-app
    ports:
      - "8080:8080"

  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
```

### Key Grafana Dashboards for Spring Boot:

- Dashboard ID **4701** — JVM (Micrometer)
- Dashboard ID **12900** — Spring Boot 3.x Statistics
- Dashboard ID **11378** — Spring Boot Hikari Connection Pool

---

## Distributed Tracing with Micrometer Tracing

```xml
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
```

```
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

```yaml
management:
  tracing:
    sampling:
      probability: 1.0      # sample 100% (use 0.1 in prod)
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

Trace IDs are automatically propagated across HTTP calls. Logs include `[traceId, spanId]`.

---

## Changing Log Levels at Runtime

```
# View current level
curl http://localhost:8080/actuator/loggers/com.example

# Change to DEBUG at runtime (no restart!)
curl -X POST http://localhost:8080/actuator/loggers/com.example \
  -H "Content-Type: application/json" \
  -d '{"configuredLevel": "DEBUG"}'
```

---

## Graceful Shutdown

```yaml
server:
  shutdown: graceful        # wait for active requests to complete

spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s   # max wait time

management:
  endpoint:
    shutdown:
      enabled: true
```

```
# Trigger graceful shutdown
curl -X POST http://localhost:8080/actuator/shutdown
```

---

### Security for Actuator

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/actuator/health", "/actuator/info").permitAll()
        .requestMatchers("/actuator/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    );
    return http.build();
}
```

Best practice in production: run actuator on a different port (not exposed to internet):

```yaml
management:
  server:
    port: 8081
```

Then firewall port 8081 from external access.

## Chapter 10: Exception Handling & Validation

### Why Centralized Exception Handling?

Without centralized handling, Spring Boot returns a generic error response. With `@ControllerAdvice`, you get consistent, informative error responses across all controllers.

**Default Spring Boot error response:**

```json
{
  "timestamp": "2024-01-15T10:30:00.000+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/api/users"
}
```

**Custom error response:**

```json
{
  "status": 404,
  "error": "User Not Found",
  "message": "No user exists with id: 42",
  "path": "/api/users/42",
  "timestamp": "2024-01-15T10:30:00.000+00:00"
}
```

---

## Standard Error Response DTO

```java
public record ErrorResponse(
    int status,
    String error,
    String message,
    String path,
    LocalDateTime timestamp,
    Map<String, String> fieldErrors  // for validation errors
) {
    public static ErrorResponse of(int status, String error, String message,
String path) {
        return new ErrorResponse(status, error, message, path,
LocalDateTime.now(), null);
    }
}
```

---

## Global Exception Handler with @ControllerAdvice

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Business exceptions
    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex,
HttpServletRequest request) {
```

```
        return ErrorResponse.of(404, "Not Found", ex.getMessage(),
request.getRequestURI());
    }

    @ExceptionHandler(DuplicateResourceException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ErrorResponse handleConflict(DuplicateResourceException ex,
HttpServletRequest request) {
        return ErrorResponse.of(409, "Conflict", ex.getMessage(),
request.getRequestURI());
    }

    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public ErrorResponse handleForbidden(AccessDeniedException ex,
HttpServletRequest request) {
        return ErrorResponse.of(403, "Forbidden", "Access denied",
request.getRequestURI());
    }

    // Validation exception — @Valid failures
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleValidation(MethodArgumentNotValidException ex,
                                          HttpServletRequest request) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            fieldErrors.put(error.getField(), error.getDefaultMessage())
        );
        return new ErrorResponse(
            400, "Validation Failed", "Request validation failed",
            request.getRequestURI(), LocalDateTime.now(), fieldErrors
        );
    }

    // @PathVariable / @RequestParam type mismatch
    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse handleTypeMismatch(MethodArgumentTypeMismatchException
ex,
                                            HttpServletRequest request) {
        String message = "Invalid value '" + ex.getValue() + "' for parameter
'" + ex.getName() + "'";
        return ErrorResponse.of(400, "Bad Request", message,
request.getRequestURI());
    }
```

```
    // Fallback for unexpected exceptions
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneral(Exception ex, HttpServletRequest
request) {
        log.error("Unexpected error", ex);   // log full stack trace
        return ErrorResponse.of(500, "Internal Server Error",
            "An unexpected error occurred", request.getRequestURI());
    }
}
```

---

## Custom Business Exceptions

java

```
// Base exception
public abstract class AppException extends RuntimeException {
    private final HttpStatus status;

    protected AppException(String message, HttpStatus status) {
        super(message);
        this.status = status;
    }

    public HttpStatus getStatus() { return status; }
}

// Specific exceptions
public class ResourceNotFoundException extends AppException {
    public ResourceNotFoundException(String resourceName, Long id) {
        super(resourceName + " not found with id: " + id,
HttpStatus.NOT_FOUND);
    }
}

public class DuplicateResourceException extends AppException {
    public DuplicateResourceException(String message) {
        super(message, HttpStatus.CONFLICT);
    }
}

public class BusinessRuleException extends AppException {
```

```
    public BusinessRuleException(String message) {
        super(message, HttpStatus.UNPROCESSABLE_ENTITY);
    }
}
```

---

## Bean Validation Annotations

```java
public record CreateUserRequest(

    @NotNull(message = "Name cannot be null")
    @NotBlank(message = "Name cannot be blank")
    @Size(min = 2, max = 100, message = "Name must be between 2 and 100
characters")
    String name,

    @NotBlank
    @Email(message = "Must be a valid email address")
    String email,

    @NotBlank
    @Size(min = 8, message = "Password must be at least 8 characters")
    @Pattern(
        regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).*$",
        message = "Password must contain uppercase, lowercase, and a digit"
    )
    String password,

    @Min(value = 18, message = "Must be at least 18 years old")
    @Max(value = 120)
    Integer age,

    @Past(message = "Birth date must be in the past")
    LocalDate birthDate,

    @Future(message = "Appointment must be in the future")
    LocalDateTime appointment,

    @Positive(message = "Price must be positive")
    BigDecimal price,

    @NotEmpty(message = "At least one tag is required")
    @Size(max = 10, message = "Too many tags")
```

```
    List<@NotBlank String> tags
) {}
```

---

## Custom Validators

java

```
// 1. Define the annotation
@Documented
@Constraint(validatedBy = UniqueEmailValidator.class)
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface UniqueEmail {
    String message() default "Email is already in use";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// 2. Implement the validator
@Component
public class UniqueEmailValidator implements ConstraintValidator<UniqueEmail,
String> {

    private final UserRepository userRepository;

    public UniqueEmailValidator(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public boolean isValid(String email, ConstraintValidatorContext context) {
        if (email == null) return true;   // let @NotNull handle null
        return !userRepository.existsByEmail(email);
    }
}

// 3. Use it
public record CreateUserRequest(
    @UniqueEmail
    @Email
    String email
) {}
```

### Class-Level Validation (cross-field):

```java
@PasswordMatch(message = "Passwords do not match")
public record ChangePasswordRequest(
    @NotBlank String newPassword,
    @NotBlank String confirmPassword
) {}

@Constraint(validatedBy = PasswordMatchValidator.class)
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface PasswordMatch {
    String message() default "Passwords do not match";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class PasswordMatchValidator implements
ConstraintValidator<PasswordMatch, ChangePasswordRequest> {
    @Override
    public boolean isValid(ChangePasswordRequest req,
ConstraintValidatorContext ctx) {
        return req.newPassword().equals(req.confirmPassword());
    }
}
```

---

## Validation Groups

Apply different validation rules for create vs update:

```java
public interface OnCreate {}
public interface OnUpdate {}

public record UserRequest(
    @NotNull(groups = OnCreate.class)   // only on create
    @Null(groups = OnUpdate.class)      // must be null on update
    String email,

    @NotBlank
    String name
) {}
```

```
// In controller:
@PostMapping
public UserDto create(@Validated(OnCreate.class) @RequestBody UserRequest req)
{ ... }

@PutMapping("/{id}")
public UserDto update(@PathVariable Long id,
                      @Validated(OnUpdate.class) @RequestBody UserRequest req)
{ ... }
```

---

## Handling Constraint Violations from Service Layer

```java
@Service
@Validated   // enables method-level validation
public class UserService {

    public UserDto findById(@NotNull @Positive Long id) {
        return userRepository.findById(id)
            .map(this::toDto)
            .orElseThrow(() -> new ResourceNotFoundException("User", id));
    }
}
```

`ConstraintViolationException` is thrown for method-level validation failures — handle it in `@ControllerAdvice`.

---

## Problem Details (RFC 7807) — Spring Boot 3+

Spring Boot 3.x supports RFC 7807 Problem Details out of the box:

```yaml
spring:
  mvc:
    problemdetails:
      enabled: true
```

This produces structured error responses automatically for standard exceptions:

```json
{
  "type": "about:blank",
  "title": "Bad Request",
  "status": 400,
  "detail": "Failed to read request",
  "instance": "/api/users"
}
```

---

### Validation Error Response Example

```
POST /api/users
{
  "name": "",
  "email": "not-an-email",
  "age": 10
}

Response 400:
{
  "status": 400,
  "error": "Validation Failed",
  "message": "Request validation failed",
  "path": "/api/users",
  "timestamp": "2024-01-15T10:30:00",
  "fieldErrors": {
    "name": "Name cannot be blank",
    "email": "Must be a valid email address",
    "age": "Must be at least 18 years old"
  }
}
```

## Chapter 11: Spring Boot with Docker

### Why Docker?

Docker packages your application with all its dependencies into a container — ensuring it runs identically on any machine: developer laptop, CI server, production.

---

## Option A: Simple Dockerfile

dockerfile

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

# Stage 2: Runtime (smaller image)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]


# Build image
docker build -t my-spring-app:latest .

# Run container
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod my-spring-app:latest
```

---

## Option B: Layered JAR (Optimized for Docker)

Spring Boot's layered JAR puts dependencies, Spring Boot libs, and app code in separate layers. Docker caches unchanged layers, making rebuilds fast.

dockerfile

```
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests
RUN java -Djarmode=layertools -jar target/*.jar extract --destination
target/extracted

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Copy layers in order from least to most frequently changed
COPY --from=builder /app/target/extracted/dependencies/ ./
COPY --from=builder /app/target/extracted/spring-boot-loader/ ./
COPY --from=builder /app/target/extracted/snapshot-dependencies/ ./
```

```
COPY --from=builder /app/target/extracted/application/ ./

EXPOSE 8080
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

---

## Option C: Cloud Native Buildpacks (No Dockerfile)

Spring Boot 2.3+ can build optimized Docker images without a Dockerfile:

```
# Build image directly (requires Docker running)
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=my-spring-app:latest

# With Gradle
./gradlew bootBuildImage --imageName=my-spring-app:latest
```

Configure in `pom.xml`:

```xml
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <image>
            <name>my-spring-app:${project.version}</name>
            <builder>paketobuildpacks/builder-jammy-base</builder>
        </image>
        <env>
            <BP_JVM_VERSION>21</BP_JVM_VERSION>
        </env>
    </configuration>
</plugin>
```

---

## Docker Compose for Local Development

```yaml
# docker-compose.yml
version: '3.8'
services:
```

```yaml
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=dev
      - DATABASE_URL=jdbc:postgresql://db:5432/mydb
      - DATABASE_USER=myuser
      - DATABASE_PASS=mypassword
      - REDIS_HOST=redis
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U myuser -d mydb"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  mailhog:
    image: mailhog/mailhog
    ports:
      - "1025:1025"   # SMTP
      - "8025:8025"   # Web UI

volumes:
  postgres_data:
```

```
# Start all services
docker-compose up -d

# Start only dependencies (run app locally)
docker-compose up -d db redis

# View logs
docker-compose logs -f app

# Rebuild app image
docker-compose up -d --build app

# Tear down
docker-compose down -v    # -v removes volumes
```

---

## Environment Variables in Spring Boot + Docker

```yaml
# application-prod.yml
spring:
  datasource:
    url: ${DATABASE_URL}
    username: ${DATABASE_USER}
    password: ${DATABASE_PASS}
```

Pass via Docker:

```
# Inline
docker run -e DATABASE_URL=jdbc:postgresql://host/db my-app

# From .env file
docker run --env-file .env my-app
```

### .env file (never commit to git):

```env
DATABASE_URL=jdbc:postgresql://db:5432/mydb
DATABASE_USER=myuser
DATABASE_PASS=secret
JWT_SECRET=your-very-long-secret-key
```

---

## JVM Tuning for Containers

Containers have memory limits — JVM must respect them:

```dockerfile
ENTRYPOINT ["java",
  "-XX:MaxRAMPercentage=75.0",
  "-XX:InitialRAMPercentage=50.0",
  "-XX:+UseContainerSupport",
  "-jar", "app.jar"]
```

`-XX:+UseContainerSupport` (default in JDK 11+) makes the JVM read cgroup limits instead of host memory.

For memory-constrained environments:

```
java -Xms256m -Xmx512m -jar app.jar
```

---

## Health Checks in Docker

```dockerfile
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1
```

Or using curl:

```dockerfile
HEALTHCHECK --interval=30s --timeout=10s \
  CMD curl -f http://localhost:8080/actuator/health || exit 1
```

---

## Multi-Stage Build Optimization Tips

```dockerfile
# Cache Maven/Gradle dependencies as a separate layer
FROM eclipse-temurin:21-jdk-alpine AS deps
WORKDIR /app
COPY pom.xml .
COPY .mvn .mvn
COPY mvnw .
```

```
RUN ./mvnw dependency:go-offline -B    # downloads all deps

FROM deps AS builder
COPY src ./src
RUN ./mvnw package -DskipTests

FROM eclipse-temurin:21-jre-alpine
# ... rest of runtime stage
```

Now `dependency:go-offline` layer is cached — only rebuilt when `pom.xml` changes.

---

## Pushing to a Container Registry

```
# Docker Hub
docker tag my-spring-app:latest myusername/my-spring-app:latest
docker push myusername/my-spring-app:latest

# Google Container Registry
docker tag my-spring-app:latest gcr.io/my-project/my-spring-app:latest
docker push gcr.io/my-project/my-spring-app:latest

# AWS ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin 123456789.dkr.ecr.us-east-
1.amazonaws.com
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/my-spring-app:latest
```

---

## Docker Image Size Comparison

| Base Image | Size | Notes |
|---|---|---|
| `eclipse-temurin:21-jdk` | ~430 MB | Full JDK — use for building |
| `eclipse-temurin:21-jre` | ~215 MB | JRE only — use for running |
| `eclipse-temurin:21-jre-alpine` | ~75 MB | Alpine-based — smallest |

| Base Image | Size | Notes |
|---|---|---|
| `gcr.io/distroless/java21` | ~60 MB | No shell — most secure |
| Spring Boot Buildpacks | ~90 MB | Optimized layers, no Dockerfile |

---

### .dockerignore

```
.git
target/
*.md
.env
.gitignore
Dockerfile
docker-compose*.yml
**/*.log
```

## Chapter 12: Spring Boot with Kubernetes

### Deploying Spring Boot to Kubernetes

This chapter covers deploying a Spring Boot application to Kubernetes, including best practices for health checks, configuration, secrets, and scaling.

---

### Basic Deployment

yaml

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-spring-app
  namespace: production
  labels:
    app: my-spring-app
spec:
  replicas: 3
```

```
  selector:
    matchLabels:
      app: my-spring-app
  template:
    metadata:
      labels:
        app: my-spring-app
    spec:
      containers:
        - name: my-spring-app
          image: myregistry/my-spring-app:1.0.0
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: url
            - name: DATABASE_USER
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: username
            - name: DATABASE_PASS
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: password
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:
```

```
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            initialDelaySeconds: 20
            periodSeconds: 5
            failureThreshold: 3
```

---

## Liveness vs Readiness Probes

Spring Boot Actuator provides separate health groups for Kubernetes probes:

```yaml
# application.yml
management:
  endpoint:
    health:
      probes:
        enabled: true     # enables /actuator/health/liveness and
/actuator/health/readiness
  health:
    livenessstate:
      enabled: true
    readinessstate:
      enabled: true
```

| Probe | Path | Meaning | Failure Action |
|---|---|---|---|
| **Liveness** | `/actuator/health/liveness` | App is alive (not deadlocked/crashed) | Restart the pod |
| **Readiness** | `/actuator/health/readiness` | App is ready to serve traffic | Remove from Service endpoints |
| **Startup** | `/actuator/health` | App has started | Wait longer before checking liveness |

```java
// Custom liveness indicator
@Component
public class DeadlockLivenessIndicator implements LivenessStateReporter {
    // Only marks DOWN if the app is truly broken (needs restart)
}
```

```
// Custom readiness indicator
@Component
public class DatabaseReadinessIndicator implements ReadinessStateReporter {
    // Marks DOWN if the app can't serve requests (DB is down)
}
```

---

## ConfigMaps for Application Config

```yaml
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-app-config
  namespace: production
data:
  application.yaml: |
    server:
      port: 8080
    spring:
      jpa:
        show-sql: false
    logging:
      level:
        root: INFO
        com.example: INFO
```

Mount as a file:

```yaml
containers:
  - name: my-spring-app
    volumeMounts:
      - name: config
        mountPath: /config
volumes:
  - name: config
    configMap:
      name: my-app-config
```

```yaml
# Tell Spring Boot to load from this path
```

```
  SPRING_CONFIG_LOCATION: /config/application.yaml
```

---

## Secrets

```
# Create from CLI
kubectl create secret generic db-secret \
  --from-literal=url="jdbc:postgresql://postgres:5432/mydb" \
  --from-literal=username="myuser" \
  --from-literal=password="supersecret"

# Or from YAML (values must be base64 encoded)
echo -n "supersecret" | base64   # c3VwZXJzZWNyZXQ=
```

yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  url: amRiYzpwb3N0Z3Jlc3FsOi8vcG9zdGdyZXM6NTQzMi9teWRi
  username: bXl1c2Vy
  password: c3VwZXJzZWNyZXQ=
```

---

## Service

yaml

```
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-spring-app
  namespace: production
spec:
  selector:
    app: my-spring-app
  ports:
    - protocol: TCP
      port: 80
```

```
    targetPort: 8080
  type: ClusterIP   # internal only; use LoadBalancer for external
```

---

## Ingress

yaml

```yaml
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-spring-app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - api.example.com
      secretName: api-tls
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: my-spring-app
                port:
                  number: 80
```

---

## Horizontal Pod Autoscaler (HPA)

yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-spring-app-hpa
```

```yaml
  spec:
    scaleTargetRef:
      apiVersion: apps/v1
      kind: Deployment
      name: my-spring-app
    minReplicas: 2
    maxReplicas: 10
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 70
      - type: Resource
        resource:
          name: memory
          target:
            type: Utilization
            averageUtilization: 80
```

---

## Graceful Shutdown in Kubernetes

```yaml
# application.yml
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
```

```yaml
# In Deployment spec
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60   # must be > timeout-per-shutdown-
phase
      containers:
        - name: my-spring-app
          lifecycle:
            preStop:
              exec:
```

```
                command: ["sleep", "5"]   # wait for load balancer to remove
pod
```

When Kubernetes sends `SIGTERM`:

1. Pod is removed from Service endpoints (stop receiving traffic)
2. `preStop` hook runs (sleep 5s)
3. Spring Boot receives SIGTERM → graceful shutdown starts
4. Active requests finish → app exits

---

## Helm Chart for Spring Boot

```
my-spring-app/
├── Chart.yaml
├── values.yaml
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── configmap.yaml
    ├── secret.yaml
    └── hpa.yaml
```

yaml

```yaml
# values.yaml
replicaCount: 2

image:
  repository: myregistry/my-spring-app
  tag: "1.0.0"
  pullPolicy: IfNotPresent

service:
  port: 80

ingress:
  enabled: true
  host: api.example.com

resources:
  requests:
```

```
      memory: "256Mi"
      cpu: "250m"
    limits:
      memory: "512Mi"
      cpu: "500m"

  env:
    SPRING_PROFILES_ACTIVE: prod

  secrets:
    DATABASE_URL: ""
    DATABASE_USER: ""
    DATABASE_PASS: ""

  autoscaling:
    enabled: true
    minReplicas: 2
    maxReplicas: 10
    targetCPUUtilizationPercentage: 70


  # Install
  helm install my-spring-app ./my-spring-app -f values-prod.yaml

  # Upgrade
  helm upgrade my-spring-app ./my-spring-app --set image.tag=1.1.0

  # Rollback
  helm rollback my-spring-app 1
```

---

## Zero-Downtime Deployments

yaml

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # allow 1 extra pod during update
      maxUnavailable: 0    # never reduce below desired replicas
```

With `readinessProbe` configured, Kubernetes won't route traffic to a new pod until it passes readiness — ensuring zero downtime.

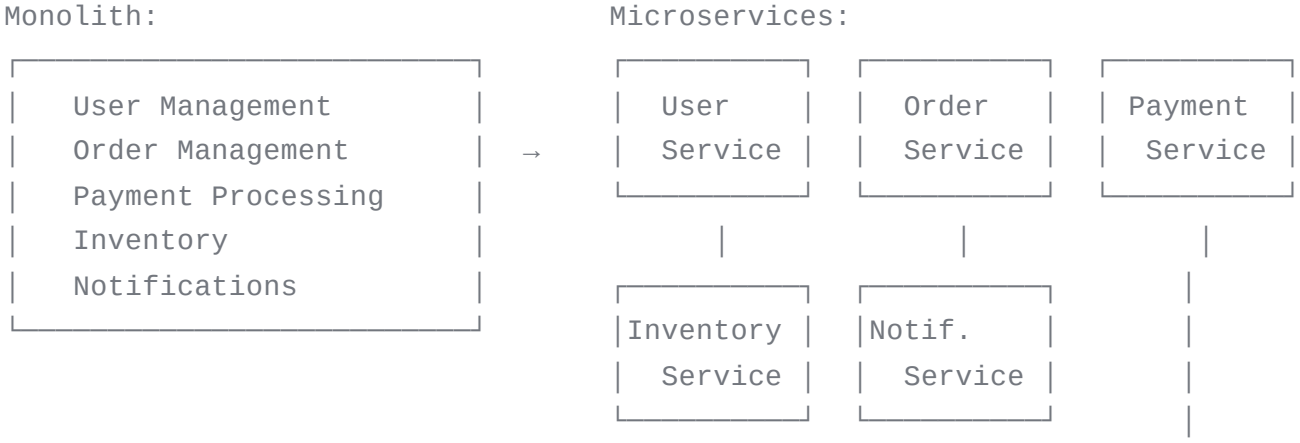
### Best Practices for Spring Boot on K8s

| Practice | Why |
|---|---|
| Always set `resources.requests` and `limits` | Prevents noisy-neighbor issues, required for HPA |
| Use `livenessProbe` and `readinessProbe` | Prevents traffic to unhealthy pods |
| Use `server.shutdown=graceful` | Prevents dropped requests during deploys |
| Store secrets in K8s Secrets or external vault | Never bake secrets into images |
| Run as non-root user | Security best practice |
| Use `readOnly: true` filesystem | Limit attack surface |
| Set `terminationGracePeriodSeconds > shutdown timeout` | Prevents forced kills |

## Chapter 13: Spring Boot Microservices

### What are Microservices?

**Microservices** is an architectural style where an application is composed of small, independently deployable services, each responsible for a specific business capability.



## When to Use Microservices

**Use microservices when:**

- Different parts of the app scale differently (payment vs catalog)
- Different teams own different domains
- Different release cycles for different features
- Technology diversity is needed (Python for ML, Java for core)

**Don't use microservices when:**

- Team is small (< 10 engineers)
- Domain boundaries aren't clear yet
- You don't have strong DevOps capabilities
- Starting a new product (start as a monolith, extract later)

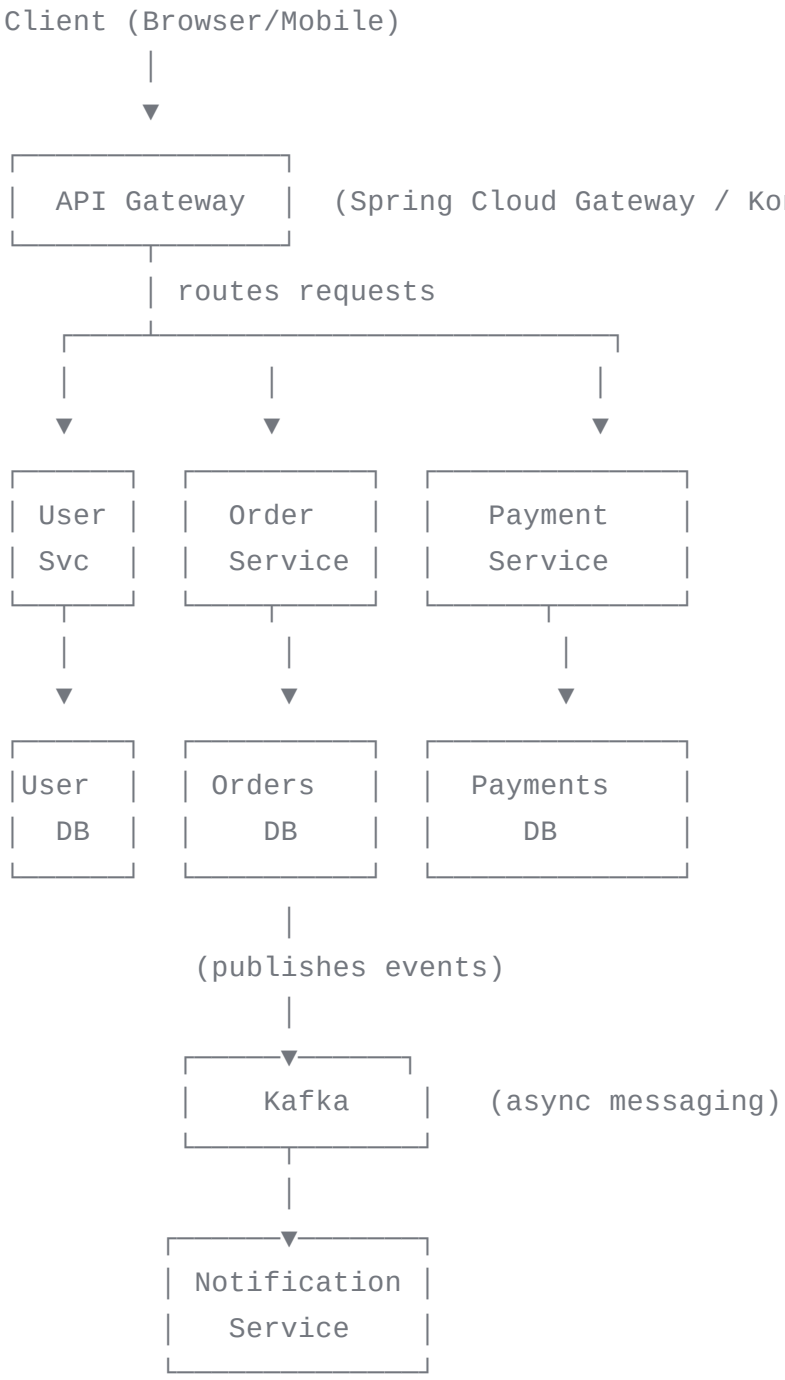
---

## Decomposition Strategies

| Strategy | How | Example |
|---|---|---|
| By business capability | Split by what the business does | Orders, Inventory, Billing |
| By subdomain (DDD) | Domain-Driven Design bounded contexts | Catalogue, Fulfillment, Customer |
| By team | Conway's Law — structure matches org | Team A → User Service, Team B → Order Service |
| Strangler Fig | Gradually extract from monolith | Start with notifications, then orders |

---

## Sample Microservices Architecture




---

## Inter-Service Communication

### Synchronous (REST / gRPC)

Use when you need an immediate response.

```java
// RestClient (Spring Boot 3.2+)
@Service
public class OrderService {
```

```
    private final RestClient restClient;

    public OrderService(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("http://user-service").build();
    }

    public UserDto getUser(Long userId) {
        return restClient.get()
            .uri("/api/users/{id}", userId)
            .retrieve()
            .body(UserDto.class);
    }
}
```

### Asynchronous (Kafka / RabbitMQ)

Use when you don't need an immediate response. Decouples services.

java

```
// Order Service publishes event
@Service
public class OrderService {
    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));
        kafkaTemplate.send("orders", new OrderCreatedEvent(order.getId(),
order.getUserId()));
        return order;
    }
}

// Notification Service consumes event
@Component
public class OrderEventListener {
    @KafkaListener(topics = "orders", groupId = "notification-service")
    public void handleOrderCreated(OrderCreatedEvent event) {
        notificationService.sendOrderConfirmation(event.getUserId(),
event.getOrderId());
    }
}
```

---

## API Gateway

The API Gateway is the single entry point for all clients. It handles:

- Routing to correct service
- Authentication (verify JWT once, not in every service)
- Rate limiting
- Request/response transformation
- Load balancing

### Spring Cloud Gateway:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service    # lb = load-balanced via Eureka
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=0
            - AddRequestHeader=X-Gateway-Source, api-gateway

        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**

        - id: rate-limit
          uri: lb://user-service
          predicates:
            - Path=/api/auth/**
          filters:
            - name: RequestRateLimiter
              args:
```

```
redis-rate-limiter.replenishRate: 10
redis-rate-limiter.burstCapacity: 20
```

---

## Service-to-Service Authentication

Each service should verify incoming requests are from trusted sources:

### Option 1: Pass JWT through

```java
// Gateway passes user's JWT to downstream services
// Downstream service validates the same JWT
```

### Option 2: Internal API Key

```java
// Each service has an internal API key
// Gateway adds X-Internal-Key header before forwarding
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/internal/**").hasRole("INTERNAL_SERVICE")
    .anyRequest().authenticated()
);
```

### Option 3: mTLS (production)

Services authenticate to each other using certificates — handled by service mesh (Istio).

---

## Database Per Service

Each service owns its own database — other services cannot query it directly.

```
User Service    → PostgreSQL (users DB)
Order Service   → PostgreSQL (orders DB)
Payment Service → PostgreSQL (payments DB)
Catalog Service → MongoDB (products DB)
Session Service → Redis
```

To share data across services:

- Expose via API (synchronous)
- Publish events to message bus (asynchronous)
- Replicate read-only data (CQRS)

---

## Saga Pattern — Distributed Transactions

Traditional ACID transactions don't work across microservices. The Saga pattern coordinates multi-step transactions:

**Choreography Saga (event-driven):**

```
Order Service   → publishes OrderCreated
                ← Inventory Service reserves stock → publishes StockReserved
                ← Payment Service charges card → publishes PaymentCompleted
                ← Fulfillment Service ships order → publishes OrderFulfilled

On failure:
Payment fails → publishes PaymentFailed
             ← Inventory Service releases stock (compensating transaction)
             ← Order Service marks order as failed
```

**Orchestration Saga (central coordinator):**

```
Order Saga Orchestrator:
1. Reserve inventory (call Inventory Service)
2. Charge payment (call Payment Service)
3. Ship order (call Fulfillment Service)
On any failure: call compensating transactions in reverse
```

---

## Shared Libraries vs Code Duplication

**Don't** share domain entities across services — this creates coupling.

**Do** share:

- Common utilities (logging format, error response model)

- OpenAPI-generated client stubs
- Security utilities

Create a shared library:

```xml
<!-- shared-lib/pom.xml -->
<artifactId>my-shared-lib</artifactId>

<!-- Each service's pom.xml -->
<dependency>
    <groupId>com.example</groupId>
    <artifactId>my-shared-lib</artifactId>
    <version>1.0.0</version>
</dependency>
```

---

## Health of the Whole System

Add readiness checks for dependencies:

```java
@Component
public class OrderServiceReadinessIndicator extends AbstractHealthIndicator {
    private final InventoryServiceClient inventoryClient;
    private final PaymentServiceClient paymentClient;

    @Override
    protected void doHealthCheck(Health.Builder builder) {
        boolean inventoryUp = inventoryClient.isHealthy();
        boolean paymentUp = paymentClient.isHealthy();

        if (inventoryUp && paymentUp) {
            builder.up().withDetail("dependencies", "all healthy");
        } else {
            builder.down()
                .withDetail("inventory", inventoryUp ? "UP" : "DOWN")
                .withDetail("payment", paymentUp ? "UP" : "DOWN");
        }
    }
}
```

---

## Key Design Principles

| Principle | What it means |
|---|---|
| Single Responsibility | Each service does one thing well |
| Loose Coupling | Services don't know each other's internals |
| High Cohesion | Related logic stays together |
| Design for failure | Assume services will fail; use timeouts, retries, circuit breakers |
| API-first | Design APIs before implementation |
| Automate everything | CI/CD, infra-as-code, automated testing |

# Chapter 14: Spring Cloud

## What is Spring Cloud?

**Spring Cloud** provides tools for building distributed systems — service discovery, centralized config, circuit breakers, distributed tracing, and more.

```xml
<!-- BOM — manages all Spring Cloud versions -->
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>2023.0.1</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

## Service Discovery with Eureka

**Eureka** lets services find each other by name instead of hardcoded host:port.

### Eureka Server:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

```java
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication { ... }
```

```yaml
# application.yml (Eureka Server)
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

### Eureka Client (each microservice):

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

```yaml
# application.yml (each service)
spring:
  application:
    name: user-service    # registered name in Eureka
eureka:
  client:
    service-url:
      defaultZone: http://eureka-server:8761/eureka/
```

```java
// RestClient with load balancing via Eureka
@Bean
@LoadBalanced    // resolves service names via Eureka
public RestClient.Builder restClientBuilder() {
    return RestClient.builder();
}
```

```
// Usage: "http://user-service" resolves to actual instance
restClient.get().uri("http://user-service/api/users/{id}",
id).retrieve().body(UserDto.class);
```

---

## OpenFeign — Declarative REST Clients

Feign generates REST client implementations from interfaces:

xml

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

java

```java
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication { ... }

@FeignClient(name = "user-service", path = "/api/users")
public interface UserServiceClient {

    @GetMapping("/{id}")
    UserDto getUserById(@PathVariable Long id);

    @GetMapping
    Page<UserDto> getAllUsers(@RequestParam int page, @RequestParam int size);

    @PostMapping
    UserDto createUser(@RequestBody CreateUserRequest request);
}

@Service
public class OrderService {
    private final UserServiceClient userClient;

    public OrderDto placeOrder(Long userId, OrderRequest request) {
        UserDto user = userClient.getUserById(userId);  // looks like a local
call!
        // ...
```

```
    }
}
```

---

## Spring Cloud Config Server

Centralize configuration for all services in a Git repository.

### Config Server:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

```java
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }
```

```yaml
# Config Server application.yml
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/myorg/config-repo
          default-label: main
          search-paths: '{application}'
```

Git repo structure:

```
config-repo/
├── application.yml           # shared by all services
├── user-service.yml          # user-service specific
├── user-service-prod.yml     # user-service in prod
└── order-service.yml
```

### Config Client (each service):

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

```yaml
# bootstrap.yml (loaded before application.yml)
spring:
  application:
    name: user-service
  config:
    import: configserver:http://config-server:8888
  profiles:
    active: prod
```

**Refresh config without restart:**

```java
@RefreshScope   // re-creates bean when /actuator/refresh is called
@Service
public class FeatureFlagService {
    @Value("${feature.new-checkout: false}")
    private boolean newCheckoutEnabled;
}
```

```
# Trigger refresh after pushing new config to Git
curl -X POST http://user-service:8080/actuator/refresh
```

---

## Resilience4j — Circuit Breaker

Circuit Breaker prevents cascade failures when a downstream service is down.

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

**States:**

```
CLOSED (normal) → 50% failures → OPEN (rejects all) → wait 60s → HALF_OPEN →
success? → CLOSED
```

java

```java
@Service
public class OrderService {

    private final CircuitBreakerFactory circuitBreakerFactory;

    public UserDto getUser(Long userId) {
        CircuitBreaker cb = circuitBreakerFactory.create("user-service");
        return cb.run(
            () -> userServiceClient.getUserById(userId),   // primary
            throwable -> getDefaultUser(userId)            // fallback
        );
    }

    private UserDto getDefaultUser(Long userId) {
        return new UserDto(userId, "Unknown", "unknown@example.com");
    }
}
```

### Annotation approach:

java

```java
@CircuitBreaker(name = "user-service", fallbackMethod = "getDefaultUser")
@Retry(name = "user-service")
@RateLimiter(name = "user-service")
@TimeLimiter(name = "user-service")
public UserDto getUser(Long userId) {
    return userServiceClient.getUserById(userId);
}

public UserDto getDefaultUser(Long userId, Exception e) {
    log.warn("Fallback for userId={}, reason={}", userId, e.getMessage());
    return new UserDto(userId, "Unknown", "");
}
```

### Configuration:

yaml

```yaml
resilience4j:
  circuitbreaker:
    instances:
      user-service:
```

```
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50        # open circuit if 50% fail
        wait-duration-in-open-state: 60s
        permitted-number-of-calls-in-half-open-state: 3
    retry:
      instances:
        user-service:
          max-attempts: 3
          wait-duration: 500ms
          retry-exceptions:
            - java.net.ConnectException
    timelimiter:
      instances:
        user-service:
          timeout-duration: 3s
```

---

## Distributed Tracing

---

### Micrometer Tracing (Spring Boot 3+):

xml

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

yaml

```
management:
  tracing:
    sampling:
      probability: 0.1   # sample 10% in production
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

Trace IDs automatically propagate through:

- RestClient / RestTemplate / WebClient
- Feign clients
- Kafka messages
- Scheduled tasks

Logs automatically include trace/span IDs:

```
2024-01-15 10:30:00 [user-service,6532d5d8e83e3aa2,6532d5d8e83e3aa2] INFO  -
Processing request
```

---

## Spring Cloud Gateway Advanced

```java
@Bean
public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("user-service", r -> r
            .path("/api/users/**")
            .filters(f -> f
                .addRequestHeader("X-Request-Source", "gateway")
                .circuitBreaker(c -> c
                    .setName("user-service")
                    .setFallbackUri("forward:/fallback/users"))
            )
            .uri("lb://user-service"))
        .build();
}

// Fallback controller
@RestController
@RequestMapping("/fallback")
public class FallbackController {
    @GetMapping("/users")
    public ResponseEntity<String> usersFallback() {
        return ResponseEntity.serviceUnavailable()
            .body("User service is temporarily unavailable");
    }
}
```

---

## Spring Cloud Components Summary

| Component | Purpose | Implementation |
|---|---|---|
| **Eureka** | Service registry/discovery | `spring-cloud-starter-netflix-eureka` |
| **Config Server** | Centralized configuration | `spring-cloud-config-server` |
| **Gateway** | API Gateway, routing | `spring-cloud-starter-gateway` |
| **OpenFeign** | Declarative REST clients | `spring-cloud-starter-openfeign` |
| **Resilience4j** | Circuit breaker, retry | `spring-cloud-starter-circuitbreaker-resilience4j` |
| **Micrometer Tracing** | Distributed tracing | `micrometer-tracing-bridge-brave` |
| **Bus** | Config refresh propagation | `spring-cloud-starter-bus-kafka` |
| **Kubernetes** | K8s-native service discovery | `spring-cloud-starter-kubernetes` |

---

## Alternative: Kubernetes-Native Discovery

Instead of Eureka, use K8s Services for service discovery:

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-kubernetes-client-all</artifactId>
</dependency>
```

Services discover each other via K8s DNS:

- `http://user-service` resolves to the K8s Service
- No Eureka server needed
- Works natively in K8s environments

# Chapter 15: Caching

## Why Caching?

Caching stores the results of expensive operations (database queries, API calls, computations) in fast storage so future requests get the cached result instead of repeating the expensive work.

```
Without cache:   Request → Service → Database (50ms) → Response
With cache hit:  Request → Service → Cache (1ms) → Response
With cache miss: Request → Service → Cache miss → Database (50ms) → Cache.store
→ Response
```

---

## Spring Cache Abstraction

Spring provides a cache abstraction that works with multiple backends: in-memory (Caffeine), Redis, Hazelcast, JCache, etc.

xml
```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>
```

java
```java
@SpringBootApplication
@EnableCaching    // activate the cache infrastructure
public class MyApplication { ... }
```

---

## Core Cache Annotations

java
```java
@Service
public class ProductService {

    // Cache the result. Key defaults to method args.
    @Cacheable(value = "products", key = "#id")
    public ProductDto getProduct(Long id) {
        log.info("Loading product {} from database", id);   // only logs on
```

```
    cache miss
            return productRepository.findById(id)
                .map(this::toDto)
                .orElseThrow(() -> new ResourceNotFoundException("Product", id));
        }

        // Evict a specific entry when a product is updated
        @CacheEvict(value = "products", key = "#id")
        public ProductDto updateProduct(Long id, UpdateProductRequest request) {
            Product product = productRepository.findById(id).orElseThrow();
            product.setName(request.name());
            product.setPrice(request.price());
            return toDto(productRepository.save(product));
        }

        // Evict ALL entries in the cache when bulk-updating
        @CacheEvict(value = "products", allEntries = true)
        public void clearProductCache() { }

        // Update cache entry after method completes (always executes the method)
        @CachePut(value = "products", key = "#result.id")
        public ProductDto createProduct(CreateProductRequest request) {
            Product product = productRepository.save(new Product(request));
            return toDto(product);
        }

        // Multiple cache operations
        @Caching(
            evict = {
                @CacheEvict(value = "products", key = "#id"),
                @CacheEvict(value = "productsByCategory", allEntries = true)
            }
        )
        public void deleteProduct(Long id) {
            productRepository.deleteById(id);
        }
    }
```

---

## Cache Keys

---

```java
// Default: all method parameters as key
@Cacheable("users")
public UserDto getUser(Long id) { ... }   // key = 1, 2, 3...

// Custom SpEL expression
@Cacheable(value = "users", key = "#email.toLowerCase()")
public UserDto getUserByEmail(String email) { ... }

// Complex key
@Cacheable(value = "search", key = "T(java.util.Objects).hash(#query, #page,
#size)")
public Page<ProductDto> search(String query, int page, int size) { ... }

// Conditional caching
@Cacheable(value = "products", condition = "#id > 0", unless = "#result ==
null")
public ProductDto getProduct(Long id) { ... }
```

---

## Option A: Simple In-Memory Cache (ConcurrentHashMap)

Works out of the box, no extra dependencies. For single-instance apps only.

```yaml
spring:
  cache:
    type: simple
```

Not suitable for production with multiple instances — each instance has its own cache.

---

## Option B: Caffeine (Local Cache with TTL)

High-performance, feature-rich in-memory cache. Best for single-instance or when Redis is overkill.

```xml
<dependency>
    <groupId>com.github.ben-manes.caffeine</groupId>
    <artifactId>caffeine</artifactId>
</dependency>
```

yaml
```yaml
spring:
  cache:
    type: caffeine
    caffeine:
      spec: maximumSize=1000,expireAfterWrite=5m
```

java
```java
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager manager = new CaffeineCacheManager();
        manager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(1_000)
            .expireAfterWrite(Duration.ofMinutes(10))
            .recordStats());   // enable cache statistics
        return manager;
    }

    // Different TTL per cache
    @Bean
    public CacheManager cacheManager() {
        SimpleCacheManager manager = new SimpleCacheManager();
        manager.setCaches(List.of(
            buildCache("products", 1000, Duration.ofMinutes(10)),
            buildCache("users", 500, Duration.ofMinutes(30)),
            buildCache("config", 100, Duration.ofHours(1))
        ));
        return manager;
    }

    private CaffeineCache buildCache(String name, int maxSize, Duration ttl) {
        return new CaffeineCache(name, Caffeine.newBuilder()
            .maximumSize(maxSize)
            .expireAfterWrite(ttl)
            .build());
    }
}
```

---

## Option C: Redis (Distributed Cache)

For multi-instance deployments — all instances share the same cache.

xml
```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

yaml
```yaml
spring:
  cache:
    type: redis
    redis:
      time-to-live: 10m        # default TTL for all caches
      key-prefix: myapp::      # prefix all keys
      use-key-prefix: true
  data:
    redis:
      host: localhost
      port: 6379
      password: ${REDIS_PASSWORD}
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 8
          max-idle: 8
          min-idle: 1
```

java
```java
@Configuration
@EnableCaching
public class RedisConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration defaultConfig =
RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))

.serializeValuesWith(RedisSerializationContext.SerializationPair.fromSerializer
(
                new GenericJackson2JsonRedisSerializer()
            ))
            .disableCachingNullValues();

        Map<String, RedisCacheConfiguration> cacheConfigs = Map.of(
            "products", defaultConfig.entryTtl(Duration.ofMinutes(30)),
```

```
            "users", defaultConfig.entryTtl(Duration.ofHours(1)),
            "sessions", defaultConfig.entryTtl(Duration.ofDays(7))
        );

        return RedisCacheManager.builder(factory)
            .cacheDefaults(defaultConfig)
            .withInitialCacheConfigurations(cacheConfigs)
            .build();
    }
}
```

---

## Redis as a Data Store (Beyond Cache)

xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

java

```
@Service
public class SessionService {

    private final StringRedisTemplate redisTemplate;

    // Store session
    public void createSession(String sessionId, UserSession session) {
        String key = "session:" + sessionId;
        redisTemplate.opsForValue().set(key, serialize(session),
Duration.ofHours(24));
    }

    // Get session
    public Optional<UserSession> getSession(String sessionId) {
        String value = redisTemplate.opsForValue().get("session:" + sessionId);
        return Optional.ofNullable(value).map(this::deserialize);
    }

    // Extend TTL
    public void extendSession(String sessionId) {
        redisTemplate.expire("session:" + sessionId, Duration.ofHours(24));
    }
```

```
    // Delete session
    public void deleteSession(String sessionId) {
        redisTemplate.delete("session:" + sessionId);
    }
}
```

---

## Cache-Aside vs Write-Through vs Write-Behind

| Pattern | How | Use When |
|---|---|---|
| **Cache-aside** (default) | App reads cache; on miss, loads from DB and stores in cache | Most read-heavy workloads |
| **Write-through** | Write to cache AND DB together | Strong consistency needed |
| **Write-behind** | Write to cache, async write to DB | High write throughput |
| **Read-through** | Cache fetches from DB on miss automatically | Simplicity |

Spring's `@Cacheable` implements cache-aside.

---

## Cache Eviction Strategies

| Strategy | Description |
|---|---|
| `expireAfterWrite` | TTL from time of write |
| `expireAfterAccess` | TTL from time of last access |
| `maximumSize` | LRU eviction when limit reached |
| Manual eviction | `@CacheEvict` or `cache.evict(key)` |

### Cache Best Practices

- Cache **read-heavy, write-rarely** data (product catalog, user profiles)
- **Don't cache** frequently changing data or user-specific sensitive data
- Set appropriate **TTL** — shorter for volatile data, longer for stable data
- **Monitor cache hit rate** — Caffeine.recordStats() or Redis INFO stats
- Use **cache warming** on startup for critical data
- Plan for **cache stampede** — use probabilistic early expiration or locking

```java
// Cache statistics with Caffeine
@Bean
public CacheManager cacheManager(MeterRegistry registry) {
    CaffeineCacheManager manager = new CaffeineCacheManager();
    manager.setCaffeine(Caffeine.newBuilder()
        .recordStats());
    // Micrometer will export cache stats automatically
    return manager;
}
```

## Chapter 16: Messaging — Kafka & RabbitMQ

### Why Messaging?

Messaging decouples services — the sender doesn't need to know if the receiver is running, how many receivers exist, or how fast they process. This enables:

- **Asynchronous processing** — respond to the user immediately, process in background
- **Load leveling** — messages queue up during spikes, consumers work at their pace
- **Event-driven architecture** — services react to events, not direct calls

---

### Apache Kafka

Kafka is a distributed, fault-tolerant event streaming platform. Best for high-throughput, event replay, and ordered processing.

### Dependency:

```xml
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

### Configuration:

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer:
org.springframework.kafka.support.serializer.JsonSerializer
      acks: all          # wait for all replicas to acknowledge
      retries: 3
      properties:
        enable.idempotence: true   # exactly-once semantics
    consumer:
      group-id: my-service
      key-deserializer:
org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
org.springframework.kafka.support.serializer.JsonDeserializer
      auto-offset-reset: earliest  # start from beginning for new consumer
groups
      enable-auto-commit: false    # manual acknowledgment for reliability
      properties:
        spring.json.trusted.packages: "com.example.events"
    listener:
      ack-mode: manual   # manually commit offset after processing
```

---

## Kafka Producer

---

```java
// Event class
public record OrderCreatedEvent(
    Long orderId,
    Long userId,
```

```
    BigDecimal totalAmount,
    List<Long> itemIds,
    LocalDateTime createdAt
) {}

@Service
public class OrderService {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    @Transactional
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));

        OrderCreatedEvent event = new OrderCreatedEvent(
            order.getId(), order.getUserId(), order.getTotal(),
            order.getItemIds(), order.getCreatedAt()
        );

        // Send async — returns CompletableFuture
        kafkaTemplate.send("orders.created", String.valueOf(order.getId()),
event)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.error("Failed to send order event: orderId={}",
order.getId(), ex);
                } else {
                    log.info("Order event sent: partition={}, offset={}",
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
                }
            });

        return order;
    }
}
```

## Kafka Consumer

java

```
@Component
public class NotificationConsumer {
```

```
    private final EmailService emailService;

    @KafkaListener(
        topics = "orders.created",
        groupId = "notification-service",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void handleOrderCreated(
            ConsumerRecord<String, OrderCreatedEvent> record,
            Acknowledgment acknowledgment) {
        try {
            OrderCreatedEvent event = record.value();
            log.info("Received order event: orderId={}", event.orderId());

            emailService.sendOrderConfirmation(event.userId(),
event.orderId());

            acknowledgment.acknowledge();   // commit offset only after
successful processing

        } catch (Exception e) {
            log.error("Failed to process order event: orderId={}",
record.key(), e);
            // Don't acknowledge — message will be redelivered
            throw e;
        }
    }

    // Listen to multiple topics
    @KafkaListener(topics = {"orders.created", "orders.updated"}, groupId =
"audit-service")
    public void auditOrderEvents(OrderCreatedEvent event) {
        auditRepository.save(new AuditLog("ORDER_EVENT", event.orderId()));
    }

    // Batch listener
    @KafkaListener(topics = "analytics.events", groupId = "analytics")
    public void processBatch(List<ConsumerRecord<String, AnalyticsEvent>>
records,
                             Acknowledgment ack) {
        analyticsRepository.saveAll(records.stream().map(r ->
r.value()).toList());
        ack.acknowledge();
```

```
    }
}
```

---

## Kafka Configuration Bean

java

```
@Configuration
public class KafkaConfig {

    @Bean
    public ConcurrentKafkaListenerContainerFactory<String, Object>
kafkaListenerContainerFactory(
            ConsumerFactory<String, Object> consumerFactory) {
        ConcurrentKafkaListenerContainerFactory<String, Object> factory =
            new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(consumerFactory);
        factory.setConcurrency(3);   // 3 threads per listener (up to partition
count)

factory.getContainerProperties().setAckMode(ContainerProperties.AckMode.MANUAL)
;

        // Dead letter queue for failed messages
        DefaultErrorHandler errorHandler = new DefaultErrorHandler(
            new DeadLetterPublishingRecoverer(kafkaTemplate),
            new FixedBackOff(1000L, 3)   // retry 3 times with 1s delay
        );
        factory.setCommonErrorHandler(errorHandler);

        return factory;
    }

    @Bean
    public NewTopic orderCreatedTopic() {
        return TopicBuilder.name("orders.created")
            .partitions(6)
            .replicas(3)
            .build();
    }
}
```

## RabbitMQ

RabbitMQ is a message broker using the AMQP protocol. Best for task queues, request-reply patterns, and complex routing.

### Dependency:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

### Configuration:

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
    virtual-host: /
    listener:
      simple:
        acknowledge-mode: manual
        retry:
          enabled: true
          max-attempts: 3
          initial-interval: 1000ms
```

### Setup Queues, Exchanges, Bindings:

```java
@Configuration
public class RabbitMQConfig {

    public static final String ORDER_QUEUE = "orders.queue";
    public static final String ORDER_EXCHANGE = "orders.exchange";
    public static final String ORDER_ROUTING_KEY = "order.created";
    public static final String DLQ = "orders.dlq";
    public static final String DLX = "orders.dlx";

    // Dead Letter Queue
```

```
    @Bean
    public Queue deadLetterQueue() {
        return QueueBuilder.durable(DLQ).build();
    }

    @Bean
    public DirectExchange deadLetterExchange() {
        return new DirectExchange(DLX);
    }

    @Bean
    public Binding dlqBinding() {
        return
BindingBuilder.bind(deadLetterQueue()).to(deadLetterExchange()).with(DLQ);
    }

    // Main queue with DLQ configured
    @Bean
    public Queue orderQueue() {
        return QueueBuilder.durable(ORDER_QUEUE)
            .withArgument("x-dead-letter-exchange", DLX)
            .withArgument("x-dead-letter-routing-key", DLQ)
            .withArgument("x-message-ttl", 300000)   // 5 min TTL
            .build();
    }

    @Bean
    public TopicExchange orderExchange() {
        return new TopicExchange(ORDER_EXCHANGE);
    }

    @Bean
    public Binding orderBinding() {
        return
BindingBuilder.bind(orderQueue()).to(orderExchange()).with(ORDER_ROUTING_KEY);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory factory) {
        RabbitTemplate template = new RabbitTemplate(factory);
        template.setMessageConverter(jsonMessageConverter());
```

```
        template.setConfirmCallback((correlationData, ack, cause) -> {
            if (!ack) log.error("Message not confirmed: {}", cause);
        });
        return template;
    }
}
```

### RabbitMQ Producer:

```java
@Service
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));

        rabbitTemplate.convertAndSend(
            RabbitMQConfig.ORDER_EXCHANGE,
            RabbitMQConfig.ORDER_ROUTING_KEY,
            new OrderCreatedEvent(order.getId(), order.getUserId())
        );

        return order;
    }
}
```

### RabbitMQ Consumer:

```java
@Component
public class NotificationConsumer {

    @RabbitListener(queues = RabbitMQConfig.ORDER_QUEUE)
    public void handleOrderCreated(
            OrderCreatedEvent event,
            Channel channel,
            @Header(AmqpHeaders.DELIVERY_TAG) long tag) throws IOException {
        try {
            notificationService.sendOrderConfirmation(event.getUserId(),
event.getOrderId());
            channel.basicAck(tag, false);          // acknowledge
        } catch (Exception e) {
            channel.basicNack(tag, false, false);  // reject, send to DLQ
```

```
            log.error("Failed to process order: {}", event.getOrderId(), e);
        }
    }
}
```

---

## Kafka vs RabbitMQ Comparison

| Feature | Kafka | RabbitMQ |
|---|---|---|
| Model | Event log (pull) | Message queue (push) |
| Message retention | Configurable (days/weeks) | Until consumed |
| Replay | Yes — rewind offset | No — consumed messages gone |
| Throughput | Very high (millions/sec) | High (tens of thousands/sec) |
| Ordering | Per partition | Per queue |
| Routing | Topics/partitions | Exchanges/routing keys |
| Consumer groups | Multiple independent groups | Competing consumers |
| Best for | Event sourcing, analytics, audit | Task queues, RPC, complex routing |

---

## Transactional Messaging (Outbox Pattern)

Ensures messages are sent **if and only if** the database transaction commits:

java

```
@Entity
public class OutboxEvent {
    @Id @GeneratedValue
    private Long id;
    private String topic;
    private String key;
```

```
    private String payload;
    private boolean sent = false;
    private LocalDateTime createdAt;
}

@Service
public class OrderService {
    @Transactional
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));

        // Save event atomically with the order
        OutboxEvent outbox = new OutboxEvent("orders.created",
            String.valueOf(order.getId()),
            objectMapper.writeValueAsString(new OrderCreatedEvent(order)));
        outboxRepository.save(outbox);

        return order;
    }
}

// Separate scheduler reads and sends outbox events
@Scheduled(fixedDelay = 1000)
public void processOutbox() {
    outboxRepository.findBySentFalse().forEach(event -> {
        kafkaTemplate.send(event.getTopic(), event.getKey(),
event.getPayload());
        event.setSent(true);
        outboxRepository.save(event);
    });
}
```

# Chapter 17: Scheduling & Async

## @Scheduled — Scheduled Tasks

Spring's `@Scheduled` annotation runs methods on a fixed schedule. Enable it with `@EnableScheduling`.

java

```
@SpringBootApplication
@EnableScheduling
public class MyApplication { ... }
```

## Scheduling Options

```java
@Component
public class ScheduledTasks {

    // Fixed delay: wait 5 seconds AFTER the previous run completes
    @Scheduled(fixedDelay = 5000)
    public void processQueue() {
        emailQueue.flush();
    }

    // Fixed rate: run every 10 seconds regardless of completion time
    @Scheduled(fixedRate = 10000)
    public void syncMetrics() {
        metricsService.sync();
    }

    // Cron expression
    @Scheduled(cron = "0 0 2 * * *")      // 2:00 AM daily
    public void dailyCleanup() {
        cleanupService.deleteExpiredTokens();
    }

    @Scheduled(cron = "0 0 9 * * MON-FRI")  // 9 AM weekdays
    public void sendDailySummary() {
        reportService.sendSummaryEmails();
    }

    // With initialDelay — wait before first run
    @Scheduled(fixedRate = 60000, initialDelay = 30000)
    public void heartbeat() {
        monitoringService.ping();
    }

    // External cron expression from config
    @Scheduled(cron = "${app.cleanup.schedule:0 0 2 * * *}")
    public void configurableCleanup() {
        cleanupService.run();
    }
}
```

**Cron Syntax:** `second minute hour dayOfMonth month dayOfWeek`

```
"0 0 * * * *"          every hour
"*/15 * * * * *"       every 15 seconds
"0 0 8-17 * * MON-FRI" 8 AM to 5 PM, Monday to Friday
"0 0 12 1 * *"         noon on the 1st of every month
"0 30 9 * * ?"         9:30 AM daily (? = any day of week)
```

---

## Thread Pool for Schedulers

By default, all `@Scheduled` tasks share a **single thread** — a slow task blocks others.

java
```java
@Configuration
public class SchedulerConfig implements SchedulingConfigurer {

    @Override
    public void configureTasks(ScheduledTaskRegistrar taskRegistrar) {
        taskRegistrar.setScheduler(scheduledTaskExecutor());
    }

    @Bean(destroyMethod = "shutdown")
    public Executor scheduledTaskExecutor() {
        return Executors.newScheduledThreadPool(10);
    }
}
```

---

## @Async — Asynchronous Execution

`@Async` runs a method in a separate thread, returning immediately to the caller.

java
```java
@SpringBootApplication
@EnableAsync
public class MyApplication { ... }
```

java
```java
@Service
public class EmailService {

    // Fire and forget — returns void
    @Async
```

```
    public void sendWelcomeEmail(String email) {
        log.info("Sending email on thread: {}",
Thread.currentThread().getName());
        // ... slow email sending logic
    }

    // Returns a Future
    @Async
    public CompletableFuture<EmailResult> sendEmail(EmailRequest request) {
        EmailResult result = mailSender.send(request);
        return CompletableFuture.completedFuture(result);
    }
}

@Service
public class UserService {
    private final EmailService emailService;

    public UserDto registerUser(CreateUserRequest request) {
        User user = userRepository.save(new User(request));
        emailService.sendWelcomeEmail(user.getEmail());  // returns immediately
        return toDto(user);
    }
}
```

**Important:** `@Async` only works when called from **outside** the bean. Self-calls (calling `@Async` method from the same class) won't work — Spring can't intercept them.

---

## Async Thread Pool Configuration

java

```
@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
```

```
        executor.setRejectedExecutionHandler(new
ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
            log.error("Async exception in {}: {}", method.getName(),
ex.getMessage(), ex);
    }
}
```

### Multiple Executor Pools:

java

```
@Bean("emailExecutor")
public Executor emailExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(3);
    executor.setMaxPoolSize(10);
    executor.setThreadNamePrefix("email-");
    executor.initialize();
    return executor;
}

@Bean("reportExecutor")
public Executor reportExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(2);
    executor.setMaxPoolSize(5);
    executor.setThreadNamePrefix("report-");
    executor.initialize();
    return executor;
}

// Use specific executor
@Async("emailExecutor")
public void sendEmail(String email) { ... }

@Async("reportExecutor")
public CompletableFuture<Report> generateReport(Long id) { ... }
```

## Combining Async and Parallel Execution

```java
@Service
public class DashboardService {

    private final UserService userService;
    private final OrderService orderService;
    private final AnalyticsService analyticsService;

    public DashboardData loadDashboard(Long userId) throws ExecutionException,
InterruptedException {
        // Run all three in parallel
        CompletableFuture<UserDto> userFuture = CompletableFuture.supplyAsync(
            () -> userService.findById(userId));

        CompletableFuture<List<OrderDto>> ordersFuture =
CompletableFuture.supplyAsync(
            () -> orderService.getRecentOrders(userId, 10));

        CompletableFuture<AnalyticsDto> analyticsFuture =
CompletableFuture.supplyAsync(
            () -> analyticsService.getSummary(userId));

        // Wait for all to complete
        CompletableFuture.allOf(userFuture, ordersFuture,
analyticsFuture).join();

        return new DashboardData(
            userFuture.get(),
            ordersFuture.get(),
            analyticsFuture.get()
        );
    }
}
```

## Virtual Threads (Java 21+)

Java 21's virtual threads (Project Loom) allow massive concurrency with simple thread-per-request code:

```yaml
# application.yml
spring:
  threads:
    virtual:
      enabled: true   # Spring Boot 3.2+ — uses virtual threads for Tomcat and
@Async
```

```java
// Or configure manually
@Bean
public AsyncTaskExecutor applicationTaskExecutor() {
    return new
TaskExecutorAdapter(Executors.newVirtualThreadPerTaskExecutor());
}
```

With virtual threads:

- Each HTTP request gets its own virtual thread (millions can exist simultaneously)
- Blocking I/O (database, HTTP) yields the carrier thread — no thread pool starvation
- No need for reactive programming for I/O-bound workloads

---

## Programmatic Scheduling

```java
@Service
public class DynamicSchedulerService {

    private final TaskScheduler taskScheduler;
    private final Map<String, ScheduledFuture<?>> scheduledTasks = new
ConcurrentHashMap<>();

    public void scheduleTask(String taskId, Runnable task, String
cronExpression) {
        CronTrigger trigger = new CronTrigger(cronExpression);
        ScheduledFuture<?> future = taskScheduler.schedule(task, trigger);
        scheduledTasks.put(taskId, future);
        log.info("Scheduled task {} with cron: {}", taskId, cronExpression);
    }

    public void cancelTask(String taskId) {
        ScheduledFuture<?> future = scheduledTasks.remove(taskId);
        if (future != null) {
```

```
            future.cancel(false);
            log.info("Cancelled task: {}", taskId);
        }
    }
}
```

---

## Transaction + @Async

`@Async` methods run in a new thread — the original transaction does **not** propagate.

java

```
@Service
public class OrderService {

    @Transactional
    public Order placeOrder(OrderRequest request) {
        Order order = orderRepository.save(new Order(request));
        // At this point transaction is open

        notificationService.sendConfirmation(order.getId());
        // This runs in a NEW thread with NO transaction
        // It will start its own transaction when it hits @Transactional
methods

        return order;
    }
}
```

---

## Best Practices

| Practice | Why |
|---|---|
| Always configure a thread pool | Default `SimpleAsyncTaskExecutor` creates a new thread per call |
| Set `setQueueCapacity` | Prevents unbounded queue memory growth |

| Practice | Why |
|---|---|
| Handle async exceptions | They're swallowed silently without `AsyncUncaughtExceptionHandler` |
| Use `CompletableFuture` over `Future` | Better composition, non-blocking API |
| Log thread names | Makes debugging async flows easier |
| Don't use `@Async` in the same class | Bypasses the proxy — won't work |

# Chapter 18: Spring Boot with GraphQL

## What is GraphQL?

**GraphQL** is an API query language that lets clients request exactly the data they need — no more, no less. Unlike REST where the server decides what's in the response, GraphQL gives clients control.

```
REST:                              GraphQL:
GET /users/1                       query {
GET /users/1/orders                  user(id: 1) {
GET /users/1/orders/5/items            id
                                       name
= 3 requests, possible overfetch       orders {
                                         id
                                         totalAmount
                                         items { name, price }
                                       }
                                     }
                                   }
                                   = 1 request, exactly what's needed
```

## Dependencies

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

---

## GraphQL Schema

Define your schema in `src/main/resources/graphql/schema.graphqls` :

graphql

```
type Query {
    user(id: ID!): User
    users(page: Int = 0, size: Int = 10): UserPage!
    searchUsers(name: String!): [User!]!
}

type Mutation {
    createUser(input: CreateUserInput!): User!
    updateUser(id: ID!, input: UpdateUserInput!): User!
    deleteUser(id: ID!): Boolean!
}

type Subscription {
    orderStatusChanged(orderId: ID!): Order!
}

type User {
    id: ID!
    name: String!
    email: String!
    createdAt: String!
    orders: [Order!]!
    orderCount: Int!
}

type Order {
    id: ID!
    totalAmount: Float!
    status: OrderStatus!
    items: [OrderItem!]!
    createdAt: String!
```

```
}

type OrderItem {
    id: ID!
    productName: String!
    quantity: Int!
    price: Float!
}

type UserPage {
    content: [User!]!
    totalElements: Int!
    totalPages: Int!
    currentPage: Int!
}

enum OrderStatus {
    PENDING
    CONFIRMED
    SHIPPED
    DELIVERED
    CANCELLED
}

input CreateUserInput {
    name: String!
    email: String!
    password: String!
}

input UpdateUserInput {
    name: String
    email: String
}
```

---

## Query Controllers

java

```
@Controller
public class UserQueryController {

    private final UserService userService;
```

```
    // Maps to Query.user
    @QueryMapping
    public UserDto user(@Argument Long id) {
        return userService.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("User", id));
    }

    // Maps to Query.users
    @QueryMapping
    public Page<UserDto> users(@Argument int page, @Argument int size) {
        return userService.findAll(PageRequest.of(page, size));
    }

    // Maps to Query.searchUsers
    @QueryMapping
    public List<UserDto> searchUsers(@Argument String name) {
        return userService.search(name);
    }
}
```

---

## Mutation Controllers

java

```
@Controller
public class UserMutationController {

    private final UserService userService;

    // Maps to Mutation.createUser
    @MutationMapping
    public UserDto createUser(@Argument @Valid CreateUserInput input) {
        return userService.create(input);
    }

    // Maps to Mutation.updateUser
    @MutationMapping
    public UserDto updateUser(@Argument Long id, @Argument UpdateUserInput
input) {
        return userService.update(id, input);
    }

    // Maps to Mutation.deleteUser
    @MutationMapping
```

```
    public boolean deleteUser(@Argument Long id) {
        return userService.delete(id);
    }
}
```

---

## Resolving Nested Fields

When a field requires a separate query (like loading `orders` for each `User`), use `@SchemaMapping`:

java

```
@Controller
public class UserFieldController {

    private final OrderService orderService;

    // Maps to User.orders — called for each User in the response
    @SchemaMapping(typeName = "User", field = "orders")
    public List<OrderDto> orders(UserDto user) {
        return orderService.findByUserId(user.id());
    }

    // Maps to User.orderCount
    @SchemaMapping(typeName = "User", field = "orderCount")
    public int orderCount(UserDto user) {
        return orderService.countByUserId(user.id());
    }
}
```

---

## The N+1 Problem and @BatchMapping

Without batching, loading orders for 10 users = 10 separate database queries (N+1 problem).

`@BatchMapping` solves this by loading all at once:

java

```
@Controller
public class UserFieldController {
```

```
    private final OrderService orderService;

    // Called ONCE with ALL users in the response — returns a Map<User,
List<Order>>
    @BatchMapping(typeName = "User", field = "orders")
    public Map<UserDto, List<OrderDto>> orders(List<UserDto> users) {
        List<Long> userIds = users.stream().map(UserDto::id).toList();
        Map<Long, List<OrderDto>> ordersByUser =
orderService.findByUserIds(userIds);

        return users.stream().collect(Collectors.toMap(
            user -> user,
            user -> ordersByUser.getOrDefault(user.id(), List.of())
        ));
    }
}
```

This turns N+1 queries into a single query.

---

## Subscriptions (Real-Time Updates)

xml

```
<!-- Additional dependency for WebSocket subscriptions -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

java

```
@Controller
public class OrderSubscriptionController {

    private final Sinks.Many<OrderDto> orderSink =
        Sinks.many().multicast().onBackpressureBuffer();

    // Maps to Subscription.orderStatusChanged
    @SubscriptionMapping
    public Flux<OrderDto> orderStatusChanged(@Argument Long orderId) {
        return orderSink.asFlux()
            .filter(order -> order.id().equals(orderId));
    }

    // Called when order status changes
```

```
    public void publishOrderUpdate(OrderDto order) {
        orderSink.tryEmitNext(order);
    }
}
```

---

## Configuration

```yaml
spring:
  graphql:
    graphiql:
      enabled: true        # enables GraphiQL browser UI at /graphiql
      path: /graphiql
    schema:
      printer:
        enabled: true
    websocket:
      path: /graphql-ws    # for subscriptions
  mvc:
    async:
      request-timeout: 60s
```

---

## Error Handling

```java
@Component
public class GraphQLExceptionHandler implements DataFetcherExceptionResolver {

    @Override
    public Mono<List<GraphQLError>> resolveException(
            Throwable exception,
            DataFetchingEnvironment environment) {

        if (exception instanceof ResourceNotFoundException ex) {
            return Mono.just(List.of(GraphqlErrorBuilder.newError(environment)
                .message(ex.getMessage())
                .errorType(ErrorType.NOT_FOUND)
                .build()));
        }
```

```
        if (exception instanceof AccessDeniedException) {
            return Mono.just(List.of(GraphqlErrorBuilder.newError(environment)
                .message("Access denied")
                .errorType(ErrorType.FORBIDDEN)
                .build()));
        }

        return Mono.empty();   // unhandled — Spring returns default error
    }
}
```

---

## Security with GraphQL

```java
@Controller
public class OrderQueryController {

    @QueryMapping
    @PreAuthorize("isAuthenticated()")
    public List<OrderDto> myOrders(@AuthenticationPrincipal UserDetails user) {
        return orderService.findByUserEmail(user.getUsername());
    }

    @MutationMapping
    @PreAuthorize("hasRole('ADMIN')")
    public boolean cancelOrder(@Argument Long id) {
        return orderService.cancel(id);
    }
}
```

---

## Testing GraphQL

```java
@SpringBootTest
@AutoConfigureHttpGraphQlTester
class UserGraphQLTest {

    @Autowired
```

```
    private HttpGraphQlTester graphQlTester;

    @Test
    void query_user_shouldReturnUser() {
        graphQlTester.document("""
            query {
                user(id: 1) {
                    id
                    name
                    email
                }
            }
        """)
        .execute()
        .path("user.name").entity(String.class).isEqualTo("Alice")

.path("user.email").entity(String.class).isEqualTo("alice@example.com");
    }

    @Test
    void mutation_createUser_shouldReturnCreatedUser() {
        graphQlTester.document("""
            mutation {
                createUser(input: { name: "Bob", email: "bob@example.com",
password: "pass1234" }) {
                    id
                    name
                }
            }
        """)
        .execute()
        .path("createUser.name").entity(String.class).isEqualTo("Bob");
    }
}
```

---

## REST vs GraphQL — When to Use

| Use REST when | Use GraphQL when |
|---|---|
| Simple CRUD resources | Complex, nested data relationships |

| Use REST when | Use GraphQL when |
|---|---|
| Public API for 3rd parties | Multiple clients (mobile, web) with different data needs |
| File uploads | Rapid UI development (client controls data shape) |
| HTTP caching is important | Reducing network round trips is critical |
| Team is already familiar with REST | Real-time subscriptions are needed |

# Chapter 19: Performance & Production Best Practices

## Performance Mindset

Production performance problems fall into a few categories:

1. **Database** — N+1 queries, missing indexes, full table scans
2. **Memory** — memory leaks, excessive GC, too-large caches
3. **Threading** — thread pool exhaustion, blocking I/O
4. **Network** — chatty microservices, large payloads
5. **Cold start** — slow JVM startup (relevant for serverless/K8s scale-to-zero)

---

## Database Performance

### 1. Connection Pool Tuning (HikariCP)

HikariCP is Spring Boot's default connection pool.

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 10      # don't blindly set high — DB can't handle unlimited
      minimum-idle: 2
      idle-timeout: 300000       # remove idle connections after 5 min
      connection-timeout: 30000  # wait max 30s for a connection
```

```
      max-lifetime: 1800000      # recycle connections every 30 min
      validation-timeout: 5000
      leak-detection-threshold: 60000  # warn if connection held > 60s
```

**Rule of thumb:** `pool-size = (number of CPU cores * 2) + effective_spindle_count`
For most apps: 10 connections is plenty. Don't go above 50.

## 2. Query Optimization

```java
// Bad: N+1 problem
List<User> users = userRepository.findAll();
users.forEach(user -> {
    List<Order> orders = orderRepository.findByUserId(user.getId());  // N queries!
});

// Good: JOIN FETCH
@Query("SELECT u FROM User u LEFT JOIN FETCH u.orders WHERE u.status = :status")
List<User> findUsersWithOrders(UserStatus status);

// Good: DTO projection (faster than loading entities + mapping)
@Query("SELECT new com.example.UserSummary(u.id, u.name, COUNT(o.id)) " +
       "FROM User u LEFT JOIN u.orders o GROUP BY u.id, u.name")
List<UserSummary> findUserSummaries();
```

## 3. Indexes

Add indexes for columns used in `WHERE`, `JOIN`, `ORDER BY`:

```sql
-- In Flyway migration
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_status_created ON orders(status, created_at DESC);

-- Composite index for pagination queries
CREATE INDEX idx_products_category_price ON products(category_id, price);
```

## 4. Pagination — Always Paginate Large Queries

```java
// Never load all records
List<User> allUsers = userRepository.findAll();  // could be millions!

// Always use pagination
Page<User> users = userRepository.findAll(PageRequest.of(0, 100,
Sort.by("createdAt").descending()));
```

### 5. Read-Only Transactions

```java
@Transactional(readOnly = true)   // skips dirty checking, faster flush
public List<UserDto> getUsers() {
    return userRepository.findAll().stream().map(this::toDto).toList();
}
```

---

## Caching Strategy

```java
// Cache expensive aggregation queries
@Cacheable(value = "dashboardStats", key = "#orgId")
@Transactional(readOnly = true)
public DashboardStats getDashboardStats(Long orgId) {
    // Heavy query with multiple joins and aggregations
    return statsRepository.computeStats(orgId);
}

// Evict on data change
@CacheEvict(value = "dashboardStats", key = "#order.orgId")
public Order placeOrder(OrderRequest request, Long orgId) { ... }
```

---

## JVM Tuning

```
# Production JVM flags (tune for your heap size)
java \
  -Xms512m \                        # initial heap
  -Xmx2g \                          # max heap
  -XX:MaxMetaspaceSize=256m \       # metaspace limit
  -XX:+UseG1GC \                    # G1 GC (good all-rounder, default JDK 9+)
```

```
    -XX:MaxGCPauseMillis=200 \        # target max GC pause
    -XX:+HeapDumpOnOutOfMemoryError \ # dump heap on OOM for analysis
    -XX:HeapDumpPath=/tmp/heapdump.hprof \
    -XX:+UseContainerSupport \        # respect container memory limits
    -XX:MaxRAMPercentage=75.0 \       # use 75% of container memory for heap
    -jar app.jar
```

### GC Options:

| GC | Flag | Best For |
|---|---|---|
| G1GC | `-XX:+UseG1GC` | Balanced latency/throughput (default Java 9+) |
| ZGC | `-XX:+UseZGC` | Ultra-low pause (<1ms), large heaps |
| Shenandoah | `-XX:+UseShenandoahGC` | Low latency (Red Hat JDK) |
| SerialGC | `-XX:+UseSerialGC` | Small footprint, single-core |

---

## GraalVM Native Image

Compile Spring Boot app to a native binary — millisecond startup, much lower memory.

xml
```
<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
</plugin>
```

```
# Build native image (requires GraalVM JDK)
./mvnw native:compile -Pnative

# Run native binary
./target/my-app
# Starts in ~100ms vs 5-10s for JVM!
```

**Trade-offs:**

- Build time: 3-10 minutes (vs seconds for JVM)
- No dynamic class loading, reflection requires configuration hints

- Profiling/debugging harder
- Best for: serverless, sidecar containers, CLI tools

---

## HTTP Performance

### Enable HTTP/2:

```yaml
server:
  http2:
    enabled: true
```

### Enable Compression:

```yaml
server:
  compression:
    enabled: true
    mime-types: application/json,application/xml,text/plain,text/html
    min-response-size: 1024   # only compress if response > 1KB
```

### Async Request Processing:

```java
// Return Callable — releases Tomcat thread while processing
@GetMapping("/reports/{id}")
public Callable<ReportDto> getReport(@PathVariable Long id) {
    return () -> reportService.generate(id);   // executed in separate thread
}
```

---

## Lazy Loading and Initialization

```yaml
spring:
  main:
    lazy-initialization: true   # beans initialized on first use, not at
```

```
startup
                                    # reduces startup time; test thoroughly
```

```java
@Bean
@Lazy   // this specific bean is lazy
public SlowToInitializeService slowService() { ... }
```

### CDS (Class Data Sharing) — Spring Boot 3.3+:

```
# Training run — capture loaded classes
java -XX:ArchiveClassesAtExit=app.jsa -jar app.jar -
Dspring.context.exit=onRefresh

# Production run — use cached class data
java -XX:SharedArchiveFile=app.jsa -jar app.jar
# 30-40% faster startup!
```

---

## Security Performance

```java
// Cache BCrypt result — BCrypt is intentionally slow
@Cacheable(value = "passwordChecks", key = "#rawPassword + #encodedPassword")
public boolean checkPassword(String rawPassword, String encodedPassword) {
    return passwordEncoder.matches(rawPassword, encodedPassword);
}
// Note: Only appropriate for rate-limited auth flows — don't cache blindly
```

---

## Observability and Profiling

### Enable JFR (Java Flight Recorder) — zero-overhead profiling:

```
java -XX:StartFlightRecording=duration=60s,filename=recording.jfr -jar app.jar
```

### Heap analysis:

```
# Trigger heap dump
curl -X POST http://localhost:8080/actuator/heapdump > heap.hprof
# Analyze with Eclipse MAT or IntelliJ
```

---

## Production Checklist

### Application:

- [ ] `server.shutdown=graceful` configured
- [ ] Health probes configured (liveness, readiness)
- [ ] Actuator on separate port, secured
- [ ] All secrets in environment variables or vault
- [ ] Structured logging (JSON) for log aggregation
- [ ] Correlation IDs / tracing enabled
- [ ] Connection pool properly sized
- [ ] Pagination on all list endpoints
- [ ] Input validation on all endpoints

### JVM:

- [ ] Heap size explicitly set ( `-Xmx` )
- [ ] `+UseContainerSupport` if running in containers
- [ ] GC algorithm chosen for workload
- [ ] OOM heap dump enabled
- [ ] JFR or async profiler available

### Database:

- [ ] All query-time columns indexed
- [ ] N+1 queries eliminated
- [ ] `readOnly=true` on read transactions
- [ ] Connection pool sized appropriately
- [ ] Database migrations with Flyway/Liquibase
- [ ] Slow query logging enabled in DB

### Infrastructure:

- [ ] Resource requests and limits set in K8s
- [ ] HPA configured
- [ ] PodDisruptionBudget to prevent full outage during deploys
- [ ] Rolling update strategy ( `maxUnavailable=0` )
- [ ] Circuit breakers for all external calls
- [ ] Retry with exponential backoff

---

## Quick Reference: Common Performance Issues

| Symptom | Likely Cause | Fix |
|---|---|---|
| Slow queries | Missing index, N+1 | Add index, use JOIN FETCH |
| High memory | Memory leak, large heap | Heap dump → MAT analysis |
| Request timeouts | Thread pool exhaustion | Increase pool, use async |
| Slow startup | Eager init, heavy beans | Lazy init, CDS |
| 503 under load | Too few pods, small pool | HPA, increase pool |
| DB connection errors | Pool exhausted | Increase pool, fix long transactions |

# Chapter 20: Learning Roadmap

## 6-Week Hands-On Plan: Zero to Production-Ready

This roadmap takes you from installing Java to deploying a production-grade Spring Boot microservice. Each week has a project milestone.

---

## Week 1: Foundation

**Goal:** Build and run your first Spring Boot REST API.

### Topics:

- Install JDK 21 and set up IntelliJ IDEA
- Use Spring Initializr to create a project
- Understand `@SpringBootApplication`, application context, DI
- Build a CRUD REST API for a simple resource (e.g., Books)
- Use H2 in-memory database with Spring Data JPA
- Run and test with curl or Postman

### Milestone Project: Library API

```
POST /api/books          { title, author, isbn, year }
GET  /api/books          (all books, paginated)
GET  /api/books/{id}
PUT  /api/books/{id}
DELETE /api/books/{id}
GET  /api/books/search?author=...
```

### Exercises:

- [ ] Create a `Book` entity with `@Entity`, `@Id`, `@Column`
- [ ] Create `BookRepository extends JpaRepository<Book, Long>`
- [ ] Create `BookService` with business logic
- [ ] Create `BookController` with all HTTP methods
- [ ] Add `@Valid` validation to create/update requests
- [ ] Handle `ResourceNotFoundException` with `@ControllerAdvice`
- [ ] Write unit tests for `BookService` with Mockito
- [ ] Write `@WebMvcTest` for `BookController`

---

## Week 2: Data, Config & Security

**Goal:** Add a real database, profiles, and JWT authentication.

### Topics:

- Connect to PostgreSQL (use Docker Compose)

- Database migrations with Flyway
- Configuration profiles (dev/prod)
- `@ConfigurationProperties` for typed config
- Spring Security + JWT authentication

### Milestone Project: Secure Library API

```
POST /api/auth/register
POST /api/auth/login      → returns JWT token
GET  /api/books           → requires authentication
POST /api/books           → requires LIBRARIAN role
DELETE /api/books/{id}    → requires ADMIN role
```

### Exercises:

- [ ] Add PostgreSQL + Flyway migrations
- [ ] Create `application-dev.yml` and `application-prod.yml`
- [ ] Create `User` entity with email and password
- [ ] Implement `UserDetailsService`
- [ ] Implement JWT generation and validation
- [ ] Add `JwtAuthFilter`
- [ ] Configure `SecurityFilterChain`
- [ ] Write integration tests with `@SpringBootTest` + Testcontainers

---

## Week 3: Production Readiness

**Goal:** Make the app observable, containerized, and deployable.

### Topics:

- Spring Boot Actuator — health, metrics, info
- Micrometer + Prometheus metrics
- Docker with multi-stage build
- Docker Compose for full local stack
- Graceful shutdown

### Milestone Project: Library API in Docker

```
# Full stack: app + postgres + redis + prometheus + grafana
docker-compose up -d

# API serves on :8080
# Prometheus on :9090
# Grafana on :3000 (JVM + HTTP dashboards)
```

### Exercises:

- [ ] Add Actuator with health, info, metrics endpoints
- [ ] Add custom `HealthIndicator` for external dependency
- [ ] Add Micrometer Prometheus registry
- [ ] Write a multi-stage Dockerfile
- [ ] Create `docker-compose.yml` with all services
- [ ] Add Redis for session caching
- [ ] Configure Prometheus scraping
- [ ] Import JVM Grafana dashboard (ID: 4701)
- [ ] Configure graceful shutdown

---

## Week 4: Caching, Messaging & Advanced Features

**Goal:** Add caching, async processing, and event-driven patterns.

### Topics:

- Spring Cache with Redis
- `@Async` and thread pools
- `@Scheduled` jobs
- Apache Kafka basics
- Exception handling patterns

### Milestone Project: Library API with Notifications

```
Book loan event → Kafka → Notification Service → Email user
                                               → Update stats (async)
Cache expensive catalog stats with Redis
Daily report generated via @Scheduled job
```

### Exercises:

- [ ] Add Redis cache for `findById` with `@Cacheable`
- [ ] Add `@CacheEvict` on update/delete
- [ ] Create a `BookLoanService` that publishes Kafka events
- [ ] Create a separate `NotificationService` (can be a different module)
- [ ] Add `@KafkaListener` for loan events
- [ ] Add `@Async` email sending
- [ ] Add `@Scheduled` daily overdue check job
- [ ] Configure proper thread pools for async work

---

## Week 5: Microservices & Spring Cloud

**Goal:** Split the monolith into microservices with service discovery.

### Topics:

- Microservices architecture patterns
- Eureka service discovery
- OpenFeign declarative clients
- Spring Cloud Gateway
- Resilience4j circuit breaker
- Distributed tracing with Zipkin

### Milestone Project: Library Platform (3 services)

```
API Gateway (:8080)
├── User Service (:8081)    → users DB
├── Book Service (:8082)    → books DB
└── Loan Service (:8083)    → loans DB
```

```
Eureka Server (:8761)
Zipkin (:9411)
```

### Exercises:

- [ ] Create Eureka Server project
- [ ] Split Library API into 3 separate services
- [ ] Register all services with Eureka
- [ ] Add Feign client in Loan Service to call User Service
- [ ] Add Spring Cloud Gateway with routing config
- [ ] Add authentication filter in Gateway
- [ ] Add Resilience4j circuit breaker to Feign client
- [ ] Configure Zipkin distributed tracing
- [ ] Verify trace propagates across services in Zipkin UI

---

## Week 6: Kubernetes Deployment

**Goal:** Deploy all services to Kubernetes.

### Topics:

- Kubernetes Deployments, Services, Ingress
- ConfigMaps and Secrets
- Health probes with Actuator
- Helm charts
- Horizontal Pod Autoscaler

### Milestone Project: Library Platform on K8s

```
# Local K8s (kind or minikube)
kubectl apply -f k8s/

# All services running in K8s
# Ingress routes traffic to correct service
# HPA scales services under load
# Secrets managed securely
```

### Exercises:

- [ ] Write Deployment YAML for each service
- [ ] Configure liveness/readiness probes
- [ ] Create K8s Secrets for DB passwords and JWT secret
- [ ] Create ConfigMaps for non-sensitive config
- [ ] Create Services and Ingress
- [ ] Package as Helm chart with `values.yaml`
- [ ] Configure HPA for Book Service
- [ ] Deploy with `helm install`
- [ ] Test rolling update: `helm upgrade --set image.tag=v2`

---

## Learning Path Visualization

```
Week 1: [Basics] → REST API, JPA, Unit Tests
    ↓
Week 2: [Security] → JWT, PostgreSQL, Profiles, Flyway
    ↓
Week 3: [Production] → Actuator, Docker, Prometheus, Grafana
    ↓
Week 4: [Async] → Redis Cache, Kafka, @Async, @Scheduled
    ↓
Week 5: [Microservices] → Eureka, Feign, Gateway, Resilience4j
    ↓
Week 6: [Kubernetes] → K8s, Helm, HPA, Probes
    ↓
[Production-Ready Spring Boot Microservice Engineer]
```

---

## Key Concepts Cheat Sheet

| Concept | Annotation/Class | Chapter |
|---|---|---|
| REST Controller | `@RestController` , `@GetMapping` | 4 |

| Concept | Annotation/Class | Chapter |
|---|---|---|
| JPA Entity | `@Entity` , `@Id` , `@Column` | 5 |
| Repository | `JpaRepository` , `@Query` | 5 |
| Transactions | `@Transactional` | 5 |
| Security | `SecurityFilterChain` , `@PreAuthorize` | 6 |
| Configuration | `@ConfigurationProperties` , `@Value` | 7 |
| Testing | `@WebMvcTest` , `@DataJpaTest` , Testcontainers | 8 |
| Health check | `HealthIndicator` , `/actuator/health` | 9 |
| Error handling | `@ControllerAdvice` , `@ExceptionHandler` | 10 |
| Validation | `@Valid` , `@NotBlank` , `@Email` | 10 |
| Docker | Dockerfile, `docker-compose.yml` | 11 |
| Kubernetes | Deployment, Service, Ingress, Helm | 12 |
| Microservices | Service decomposition, Saga pattern | 13 |
| Service discovery | Eureka, `@LoadBalanced` | 14 |
| Circuit breaker | `@CircuitBreaker` , Resilience4j | 14 |
| Caching | `@Cacheable` , `@CacheEvict` , Redis | 15 |
| Kafka | `KafkaTemplate` , `@KafkaListener` | 16 |
| RabbitMQ | `RabbitTemplate` , `@RabbitListener` | 16 |
| Async | `@Async` , `CompletableFuture` | 17 |
| Scheduling | `@Scheduled` , cron expressions | 17 |
| GraphQL | `@QueryMapping` , `@MutationMapping` | 18 |
| Native image | GraalVM, `native:compile` | 19 |

## Recommended Resources

- **Official Docs:** docs.spring.io — always start here
- **Spring Guides:** spring.io/guides — step-by-step tutorials
- **Baeldung:** baeldung.com — practical Spring Boot examples
- **Spring Initializr:** start.spring.io — project generator
- **Books:** "Spring Boot in Action" (Craig Walls), "Cloud Native Spring in Action" (Thomas Vitale)
- **GitHub:** github.com/spring-projects/spring-boot/tree/main/spring-boot-tests — official test examples

---

## Common Mistakes to Avoid

1. **Field injection** ( `@Autowired` on field) — use constructor injection
2. **Exposing @Entity in API** — always use DTOs
3. **Missing** `@Transactional` — service layer mutations need it
4. **N+1 queries** — use JOIN FETCH or @BatchMapping
5. `ddl-auto=update` **in production** — use Flyway
6. **Secrets in application.properties** — use environment variables
7. **Single thread for** `@Scheduled` — configure a thread pool
8. `@Async` **self-call** — won't work without proxy
9. **Infinite recursion in JSON** — use `@JsonIgnore` or DTOs for bidirectional relationships
10. **No timeout on external calls** — always set connection and read timeouts